



Heat Production Optimization

Semester 2, Year 2026

Students: David Avramov `daavr25@student.sdu.dk`
Gintaras Zulys `gizul25@student.sdu.dk`
Leon Jürgen Thye `lethy25@student.sdu.dk`
Nadia Kristensen `nkris25@student.sdu.dk`
Oliwia Roksana Moskalik `olmos25@student.sdu.dk`
Yaroslav Savenko `yasav25@student.sdu.dk`

Supervisor: Alan Sieradzki

Lecturers: Adam Alami
Amir Ghomashi

University of Southern Denmark
June 1, 2026

Contents

Abstract	v
1 Introduction	1
1.1 Project Objectives and Requirements	1
1.2 Problem Statement	1
2 Scrum Implementation	3
2.1 Scrum Framework	3
2.2 Scrum Roles	3
2.3 Meetings and Ceremonies	4
2.4 Tools Supporting Scrum	4
2.5 Adjustments to Scrum	4
3 Sprints	6
3.1 Plan for Sprints	6
3.2 Sprint 1	6
3.3 Sprint 2	9
3.4 Sprint 3	12
3.5 Sprint 4	14
3.6 Sprint 5	17
3.7 Sprint 6	20
4 Solution (Technical Aspect)	26
4.1 Package Diagram	26
4.2 Class Diagram	27
4.3 Activity Diagram	28
4.4 Sequence Diagram	30
4.5 Overall Technical Architecture	31
5 Implementation and Technical Realization	35
5.1 Frontend Implementation	35
5.2 Backend Implementation	38
5.3 Data Management	39

6	Discussion & Reflection	41
6.1	Scrum	41
6.2	Requirements Engineering	44
6.3	Refactoring	46
6.4	Code Review	46
6.5	Testing	47
6.6	Overall Group Reflection	48
7	Conclusion	50
	AI Usage	54
A	Team Contributions	55
B	Meeting Logs	66
C	Contracts	111
D	Code Authorship and Review	118
E	Table of Requirements	120

List of Figures

3.1	Overview of the Jira board at the start of Sprint 1	7
3.2	Overview of the Jira board at the end of Sprint 1	8
3.3	Overview of the Jira board at the start of Sprint 2	10
3.4	Overview of the Jira board at the end of Sprint 2	11
3.5	Overview of the Jira board at the start of Sprint 3	13
3.6	Overview of the Jira board at the end of Sprint 3	13
3.7	Overview of the Jira board at the start of Sprint 4	15
3.8	Overview of the Jira board at the end of Sprint 4	16
3.9	Overview of the Jira board at the start of Sprint 5	18
3.10	Overview of the Jira board at the end of Sprint 5	19
3.11	Overview of the Jira board at the start of Sprint 6	22
3.12	Overview of the Jira board at the end of Sprint 6	23
4.1	Package Diagram	26
4.2	Class Diagram	28
4.3	Activity Diagram	29
4.4	Sequence Diagram	30
5.1	Frontend Diagram	36

List of Tables

B.1	Meeting Logs	66
E.1	Functional Requirements	121
E.2	Non-Functional Requirements	123

Abstract

Efficient heat production and energy management are becoming more important in today's district heating systems, especially when operational costs, electricity prices, and energy needs to keep changing. The project focuses on creating an application that improves the scheduling of heat production across several production units. The system looks at how these production units are set up, what the heat needs are, and the conditions of the electricity market to figure out the most affordable way to produce energy in various operating situations.

The project was done using the Scrum framework and by following steps to develop software incrementally. The team has used several sprints to organize tasks, manage work together and improve through meetings, reviews, and retrospectives. In addition to project management activities, the team focused on software design and system architecture.

The report presents the entire development process, which starts with the project background, objectives and requirements, and is followed by the Scrum implementation and sprint activities. It then describes how the solution is built by using UML diagrams, architectural descriptions, frontend and backend implementation details and data management strategies. In addition, teamwork, testing, refactoring, code reviews, and requirements engineering are discussed to evaluate both the technical aspects of the project and the effectiveness of team collaboration.

By exploring the different stages of development and the decisions made during the process, the reader learns how the adoption of Scrum and software engineering techniques have been used in this project to develop a functional optimization system for district heating production.

Chapter 1

Introduction

1.1 Project Objectives and Requirements

The goal of the project is to define heat schedules for all available production units with the lowest possible cost and the highest profit. For the development, the Scrum framework was adopted. Communication between the team and product owners was established early on to provide a better feedback loop and an opportunity for thorough requirement elicitation, which were compiled in a table (Appendix E), and those prescribed in two scenarios. The user should be able to see visual representation of all relevant data, assets, optimization results, as well as being able to configure each individual asset, optimizer itself and export the results in a convenient format. The system shall ensure that heat demand is met at any time, calculate net cost for each production unit and prioritize each unit based on it, create separate result data for each production unit and calculate the optimized heat schedule.

1.2 Problem Statement

HeatItOn utility company is supplying heat to a single district heating network with nearly two thousand buildings. They utilize different units for heat production, such as traditional gas boilers or units that either consume or produce electricity. At the moment, the company does all its scheduling of units manually. However, manual scheduling is both time consuming and prone to human error. Managing all the units, constantly following the electricity prices that change hourly, and ensuring that the heat demand is met for all the buildings on the grid makes it harder to optimize the

production for the best profit. Moreover, the time spent on this task could be better utilized on other, higher-value activities.

An automated system for optimizing the production for profit can significantly speed up the process, save resources, improve cost, and optimize the schedule. This would ensure that the heat production is as profitable as it can be and improve accuracy by reducing human error. The project aims to develop a heat optimization software that can take in all the units and their data and calculate a unit schedule that will be optimized for the highest profit. It takes into consideration the hourly shifting electricity prices and the different heating seasons to make the best and most profitable choices for the unit schedule.

Chapter 2

Scrum Implementation

2.1 Scrum Framework

The project was carried out using the Scrum framework to help with iterative development, teamwork and ongoing progress while putting the heat production optimization system into action. The work has been split into six sprints and for each sprint a new Scrum Master has been assigned. This change has been decided by the team to make sure that each member got the chance to gain experience in sprint coordination and Agile software development.

2.2 Scrum Roles

The Scrum Master was in charge of organizing meetings, preparing agendas for supervisor meetings, writing meeting logs and making sure that Scrum practices were followed during each sprint. This included monitoring ticket progress, spotting tasks that were stuck, making sure that sprint goals were achieved and ensuring that the improvements from retrospectives were implemented. The team worked together on tasks, sprint planning and time estimation. The team had two supervisors - the main supervisor and one other backup supervisor. The main supervisor gave feedback every week and the substitute supervisor helped out whenever it was necessary.

2.3 Meetings and Ceremonies

The team held regular Scrum meetings, which included Sprint Planning, Daily Stand-ups, Sprint Reviews and Sprint Retrospectives. The Sprint Planning involved breaking down tasks and estimating them while using Planning Poker, which helped the team to collaboratively evaluate task complexity and workload. Sprint Reviews took place to present completed work and receive feedback from the supervisor and stakeholders. Following each review the team operated with Sprint Retrospectives to reflect on the entire sprint, spot problems and find ways to improve in the next sprint. Scrum meetings took place every Monday and Wednesday, usually in Løkken. These meetings included Daily Stand-Ups where team members shared their progress, problems and were assigned new tasks. When in-person meetings were not possible, the team decided to have meetings online using Discord [1]. In addition to Scrum ceremonies, every Friday the team had meetings with the main supervisor either online or in-person. During these meetings the team's progress, problems were discussed and feedback was provided.

2.4 Tools Supporting Scrum

Jira [2] was used as the main software development workflow tool to manage the product backlog, keep in check the progress and organize tasks. Jira was chosen over Trello [3] because it offers product and sprint backlogs, tracking sprints and better workflow management in larger projects.

Excalidraw [4] was used mainly for decision making and meeting logs. Also the team used it to create visual maps of the system's setup, workflow and planning choices, which helped the team understand things better.

2.5 Adjustments to Scrum

Even though the project stuck to the main Scrum ideas, some changes were made to suit the university setting and the limits of the team. The length of the sprint and the amount of work were changed to fit with university deadlines and exam times. The role of Scrum Master changed with each sprint, passing from one team member to another, rather than staying with one person. Some Scrum meetings were merged or made shorter when there were scheduling problems. We used communication tools

like Discord along with face-to-face meetings to help work together remotely when needed. These changes made sure that Scrum stayed useful and efficient for a student development project.

Chapter 3

Sprints

3.1 Plan for Sprints

The team's sprint plan was to have six sprints - each two weeks long. The plan was to use Sprint 0 for setting up tools, planning and designing the application. Work on Scenario 1 during Sprint 1 and finish it in Sprint 2. Start with the report and Scenario 2 in Sprint 3, and finish Scenario 2 in Sprint 4. Sprint 5 was supposed to be the start of advanced features implementation and Sprint 6 for finishing extra features, polishing up the project, and finishing the report. However, around Sprint 2, sprint goals blended and the rough time ranges for features have moved very significantly due to longer time needed to develop features than expected, and different features being implemented at a greatly different pace.

3.2 Sprint 1

Summary

Previous Sprint 0 focused on setting up the tools and getting familiarized with the group and the project, and as such Sprint 1 was focused on building a programming foundation to work on and to have clear project requirements.

Deliverables

Product increment for Sprint 1 was written high-level and low-level requirements, functional and non-functional requirements, use case diagram and user stories. In terms of programming, SDM, RDM, AM and main navigation layout were implemented.

Sprint Planning

The sprint goal was to implement the major modules: AM, SDM, RDM and navigation layout. From our Product Backlog, we added all the tasks to the Sprint Backlog that needed to be done for our Sprint goals. Planning Poker was used for task estimation.

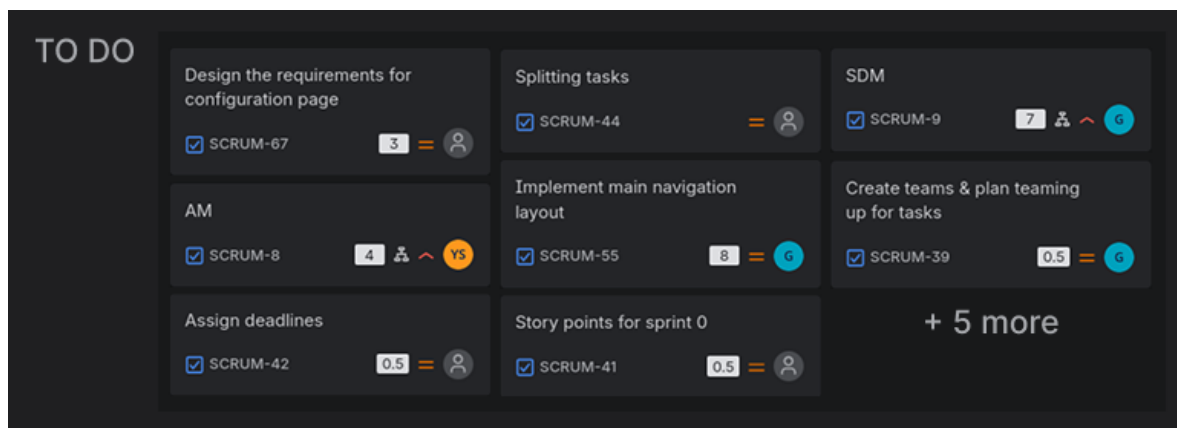


Figure 3.1: Overview of the Jira board at the start of Sprint 1

Sprint Review

When the sprint ended, almost every single task got completed, except just one task that was not completed because we found out it was not needed. Story points estimates from Sprint Planning were not realistic. Furthermore, code reviews took a considerable amount of time and slowed the progress more than expected.

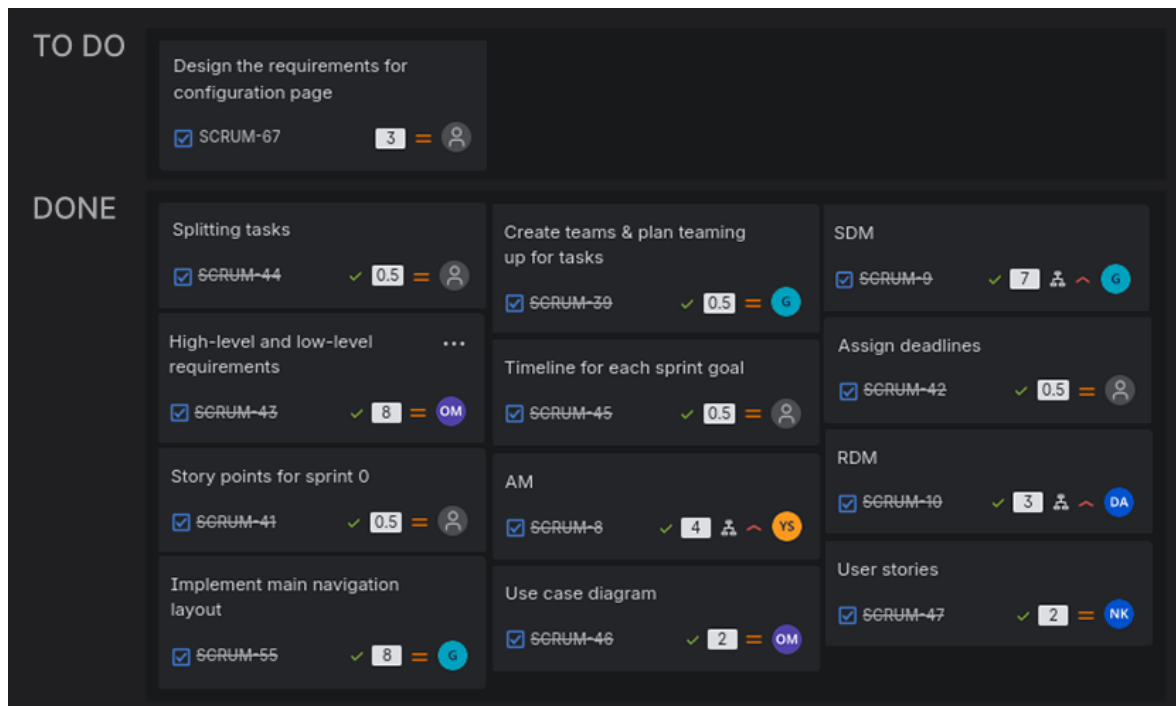


Figure 3.2: Overview of the Jira board at the end of Sprint 1

Sprint Retrospective

We did not overcommit, as we managed to deliver what was planned. Moreover, we created a solution of using an online Excalidraw canvas for decision-making to be more transparent and to encourage everyone to share their ideas. Lastly, we integrated reviews which is great for awareness and transparency, despite it taking more time.

We had issues with Jira's features and assignment of tasks. We have wanted to assign more than one team member for the task in Jira by using team assignment feature. However, the team assigned did not show up for the task on the board - it was only visible when previewing details of the task. Furthermore, subtasks are not easy to distribute and splitting tasks requires a lot of effort due to interdependencies, and there is no ability to assign someone to review the tasks.

Our team's story point estimate units did not match the reality, as tasks took about twice as long as expected. Therefore, there was not as much progress in the first week of the sprint. Moreover, members updated the Jira board with delay.

Distribution and assigning tasks during the Sprint Planning also came with some issues. Some tasks were assigned to people without asking them about it first. There was also some uneven distribution of tasks based on our story points, but in reality the estimates were not as accurate.

For the next sprint, we want to improve our task assigning both in person and in Jira. For the in-person assigning, we want to have more discussion on what each task is about and how it can be completed. Additionally, we will focus on assigning tasks formed as questions not statements, so that people do not feel forced to do tasks. On Jira, we will use labels to assign pairs as well as reviewers. The tasks will also have better descriptions so that they are clearer.

Moreover, we want to improve the progression of tasks. As tasks took longer time to complete than expected, we want to have higher transparency with why it is happening. Therefore, we will have better questions during Daily Stand-Ups:

“If not progressing as estimated, what is in the way of the progress? Be specific.”

“What was your time allocation for the task since the last meeting?”

Reflections and Conflicts

The sprint goal was reached and tasks progressed very well. As for the conflicts, for about the first week of the sprint team engagement was relatively low, and team members left unheard and their opinions ignored as they were feeling pressured and sometimes were directed to pick specific tasks. This was very prominent during time crunch situations where there was not enough time to do the tasks. In the middle of the sprint, the issue was resolved by having a shared online Excalidraw canvas during in-person meetings where tasks were listed for the session. This reduced time pressure to allocate a person to the task, team members were able to voluntarily pick tasks knowing what is needed to do, and as such team engagement and working environment improved. It worked well enough that online Excalidraw canvas usage was even further expanded in the later sprints for in-person meetings.

3.3 Sprint 2

Summary

This sprint was for learning how big the workload could be for a sprint and learning where our current communication could be improved in the group. The scope was intentionally bigger, by implementing the pages from a Figma [5] prototype with backend support.

Deliverables

The Sprint goal was to get the program finished, visual and functional for Scenario 1. The increment during this sprint was implementation of frontend of the overview page, optimizer page, production unit page and configuration page. Furthermore, implementation of Data Manager in the backend was completed. All that was done in the first half of the sprint. The second half was spent on connecting the frontend with the backend, which mostly involved working with data bindings.

Sprint Planning

This sprint was an experiment for how much work could be done in a sprint. We decided to make a big leap by planning to implement most of the UI and the required connection to connect frontend and the backend. For the implementation of the connection, Data Manager class was additionally designed and it required to be implemented this sprint. Most of the UI was prototyped in the last sprints, therefore only two pages needed to be finished with prototyping before implementation. There were quite a lot of dependent tasks (see Figure 3.3), and they tested how great our group communication is.

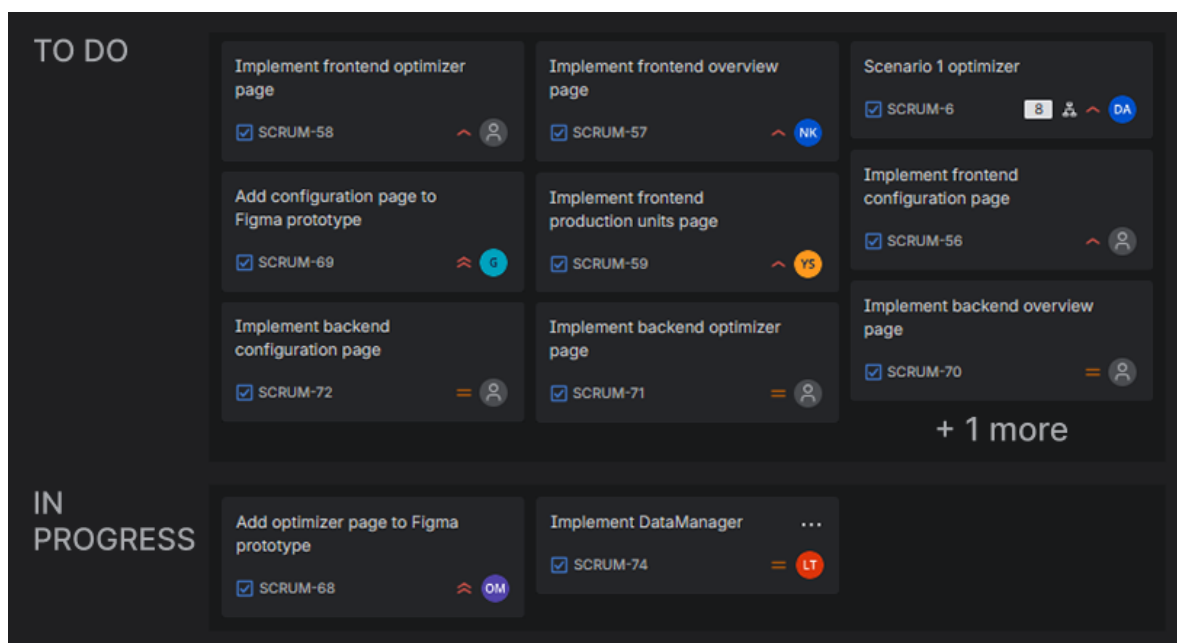


Figure 3.3: Overview of the Jira board at the start of Sprint 2

Sprint Review

At the end of this sprint, most of the UI implementation was done or in review. The backend connection tickets got stuck because they were dependent on their frontend counterparts. We had some communication issues that also contributed to the delay of some tickets, especially those in the “In Progress” section (see Figure 3.4). From the timing perspective, it was possible to get all tickets done.

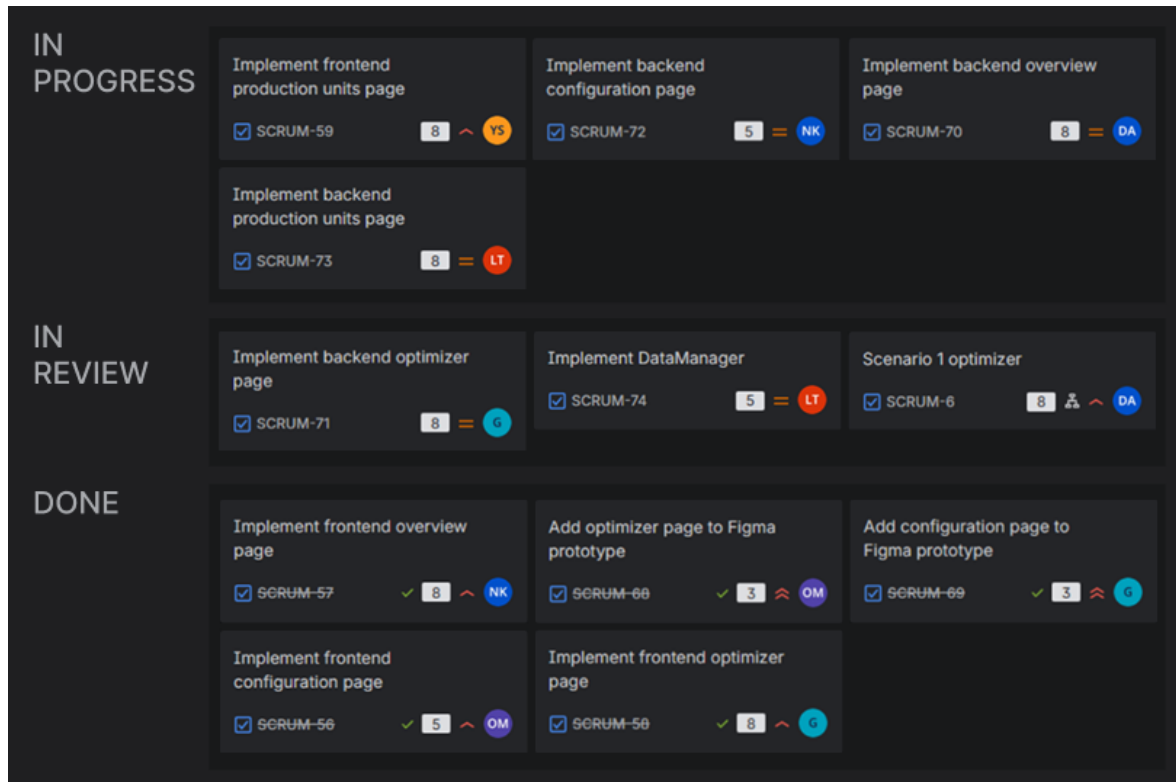


Figure 3.4: Overview of the Jira board at the end of Sprint 2

Sprint Retrospective

Because of underestimations and a misunderstanding by the Scrum Master, there were no written logs from the Sprint Retrospective. Communication happened mostly in the Meetings, and there were rarely any impediments noticed, even if we decided to be more specific in the Stand-Up meetings. The members seemed to be shy about their work and problems, not saying anything more in the meetings. One team member had problems with completing their task correctly, because they implemented the frontend without the backend. This ticket got delayed until the next sprint.

Reflections and Conflicts

To resolve the communication issues we talked with each other, and improved our Excalidraw board for writing down issues, making it easier for shy team members. The next Scrum Master will also get a task list of what he or she needs to do for a sprint, so the role and its responsibilities are more clear. A member has now the options, if they have issues with their task taking too long, to publish early the branch that he or she is working on, to write down the issue on Discord or Excalidraw board, or to talk to a team member or the current Scrum Master. We have decided to split larger tasks to smaller ones to spot issues earlier. Also, we made it clear that no one needs to be scared of each other, because of “bad code” or any other reasons.

3.4 Sprint 3

Summary

The main focus of Sprint 3 was to finish Scenario 1 for the Optimizer. Additionally, parts of backend and frontend were finalized with general connections established. For better convenience, export of the optimization results from RDM was implemented using JSON instead of CSV.

Deliverables

Delivered product increment contained Scenario 1 Optimizer implementation, frontend and backend for Production Units page, and additional expansion for RDM export.

Sprint Planning

The sprint goal was to implement the connection between backend and frontend parts, implementing export from RDM. The main goal was to reach Scenario 1. The sprint consisted of uncompleted tasks from the last sprint.

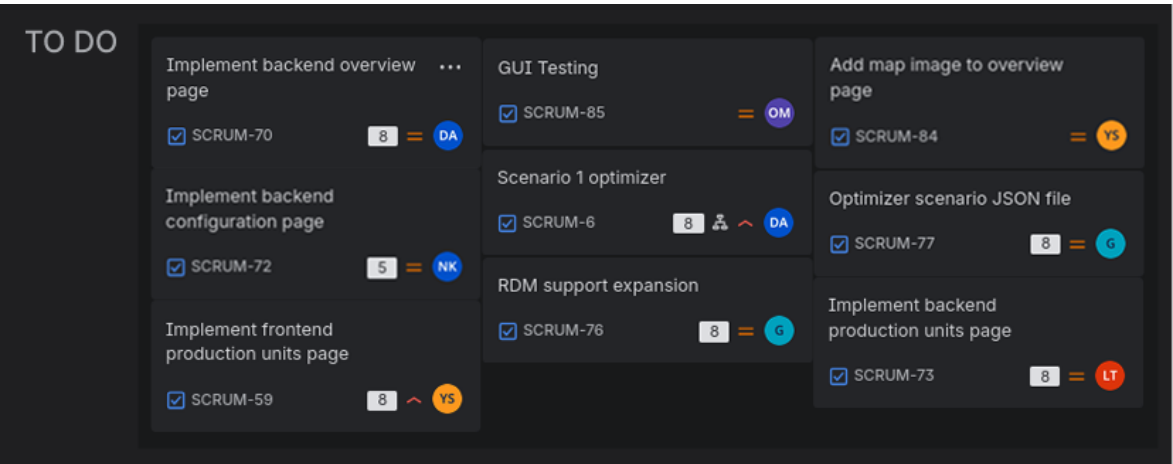


Figure 3.5: Overview of the Jira board at the start of Sprint 3

Sprint Review

The outcome of the sprint was quite mixed. Compared to the last sprints the team did not finish as many tasks. This could have been a result of the distribution of tasks between members, some of which are not as experienced as others, as well as the over-reliance of dependencies of other tasks.

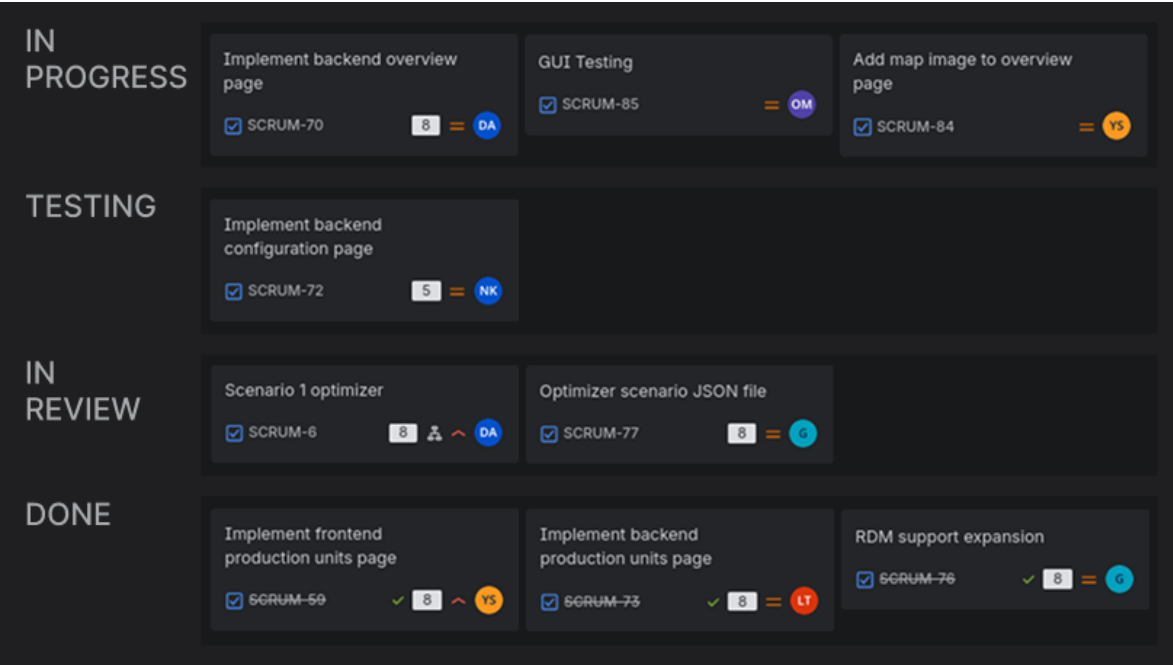


Figure 3.6: Overview of the Jira board at the end of Sprint 3

Sprint Retrospective

Based on the previous retrospective, an additional approach was taken, where to bring more transparency and streamline the process each team member would be required to notify others about the current state of their respective tasks through Discord messages. In case of problems, each member could also contact the Scrum Master or ask for help from others. The problematic part was the task planning and structure of the meetings, slow progression of tasks and missed deadlines. To resolve those issues, it was decided to move the meetings in case of external factors, asking others if they need help more frequently and requiring more responsiveness from people in case of online or hybrid meetings.

Reflections and Conflicts

A lot of the tasks were accumulated from the end of the previous sprint, and due to the easter break overlap with one of the meetings the Sprint Planning was a bit more rushed. Missed deadlines could be the result of bad task distribution and not addressing equally each of the members capabilities due to less developed communication practices throughout previous sprints.

Although the sprint ceremonies were subject to scheduling difficulties, with more confusion as to which format to use being raised by late absence notices to in-person meetings, the results of this sprint showed more transparency, communication improvements, and more elaborate attempts to collaborate on resolving arising impediments.

3.5 Sprint 4

Summary

The sprint goal for this sprint was set to finalize the implementation of Scenario 2, improve the visualization of the application, and to begin working on the report.

Deliverables

The product increment during this sprint included merging the production unit page with the configuration page, finishing all backend implementation for the UI, improving the optimizer, and adding layered charts and heat grid map to the overview page.

Sprint Planning

For the Sprint Planning in Sprint 4, we decided to try a new technique for assigning tasks. Everyone pre-estimated the amount of time they expected to have for the project during the two weeks, as we expected it to make the members feel more obligated to complete their tasks. After playing Planning Poker, all members selected their tasks on Jira based on their own work estimation and the corresponding story points on the tickets. The Sprint Backlog contained a few tasks from the previous sprint that were in the final stages and were completed in between our sprints, therefore we started with tickets in the done column (see Figure 3.7).

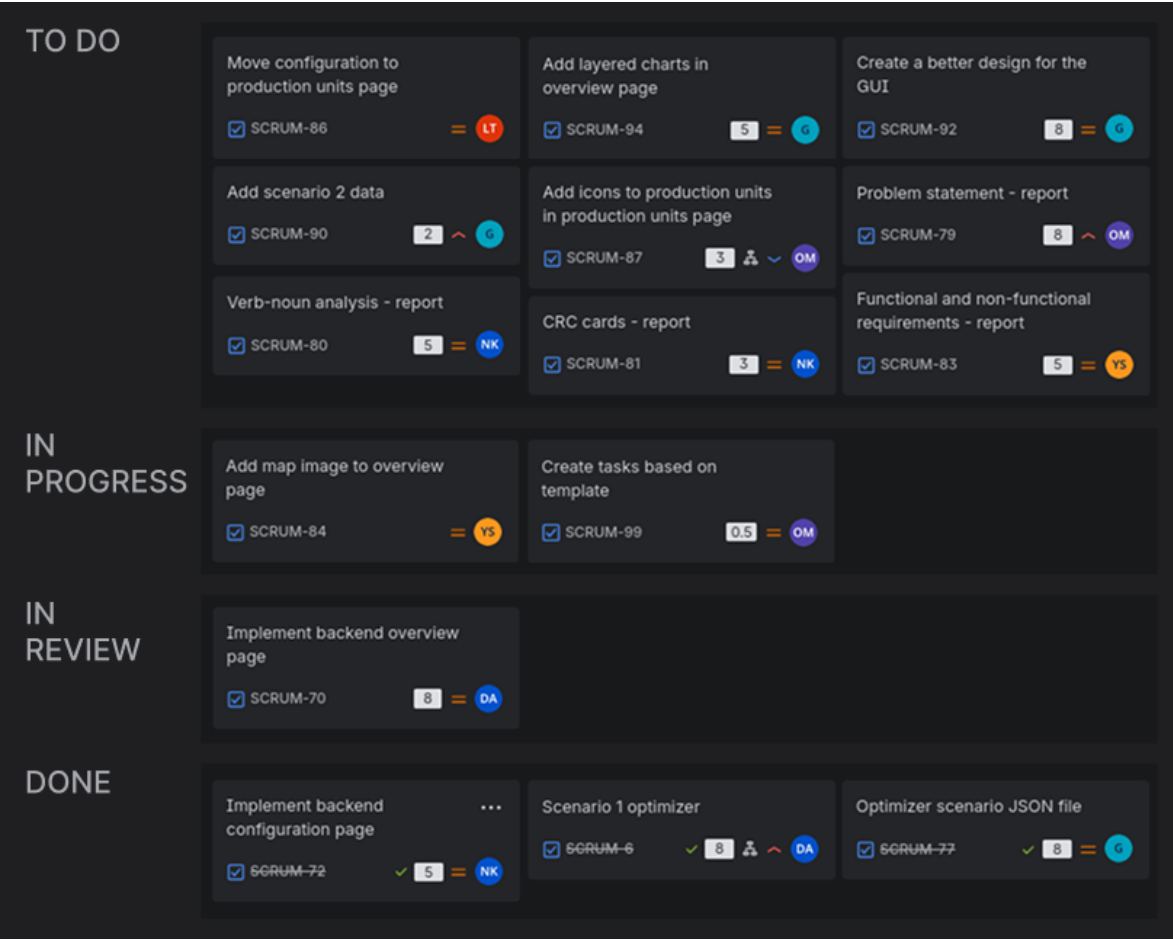


Figure 3.7: Overview of the Jira board at the start of Sprint 4

Sprint Review

At the end of the sprint, most of the tasks were completed or in review (see Figure 3.8), with two tasks in progress and only one not started. The increment of the product did not fully satisfy the sprint goal, however we progressed a lot towards finishing the application. The Sprint 4 demonstration was moved to a later date, therefore we did not receive feedback from the stakeholders at the end of the sprint.

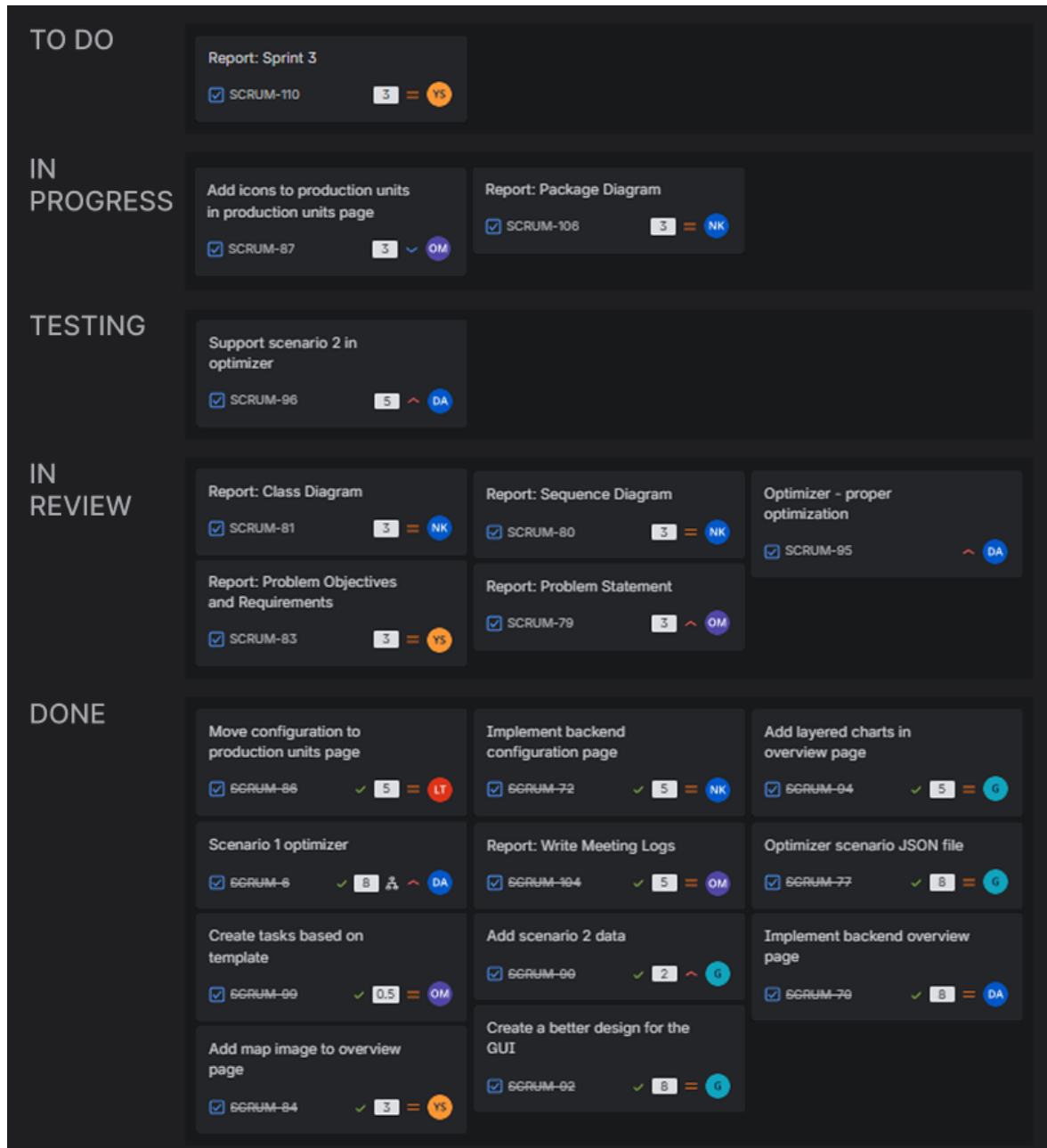


Figure 3.8: Overview of the Jira board at the end of Sprint 4

Sprint Retrospective

The Sprint Retrospective started with each member writing what worked well and what did not on our Excalidraw board, of which summary can be found in the meeting logs in the Appendix B. Everyone agreed that the tasks are progressing fairly well, and that implementation of more frequent communication over text messages is working. However, the issue of poor attendance was brought up. It was common that the meetings had only three to four members attending. Combined with the team members often getting late, the meetings seemed like they were optional and this created some discouragement, as the communication around the attendance was also quite poor.

Reflections and Conflicts

To resolve the issues, we first identified the reason behind team members getting late or being absent, which turned out to be an issue with oversleeping. We came up with a solution to move the time of the meeting to a later hour to see if that will improve the attendance. Moreover, during the discussion another issue was found. Some members felt that the ending of the meetings was not clear enough, and to solve it we decided to have a question if everyone is ready to end the meeting.

3.6 Sprint 5

Summary

During this sprint we improved the optimizer by adding new Scenario 2 features such as support for emissions and consumptions, and handling of negative prices. The UI was also improved and the codebase was refactored to better support multiple scenarios and data handling. Additionally, we worked on the report by updating sections and handled ticket and pull request reviews to support overall project quality.

Deliverables

At the end of the sprint, we delivered an improved optimizer with Scenario 2 features, a refined UI with better data and scenario support, and a refactored codebase supporting multiple scenarios. In addition, updated documentation and report sections, along with

improved diagrams and reviewed pull requests, were completed.

Sprint Planning

The sprint goal was to work on different parts of the report, including the sprint implementation, technical architecture, abstract, Scrum implementation, problem statement, diagrams, and the problem objectives and requirements. Additionally, some coding tasks were assigned, such as GUI testing and supporting implementation-related improvements. Overall, the focus was on completing both documentation and technical components to ensure consistency and final integration of the project.

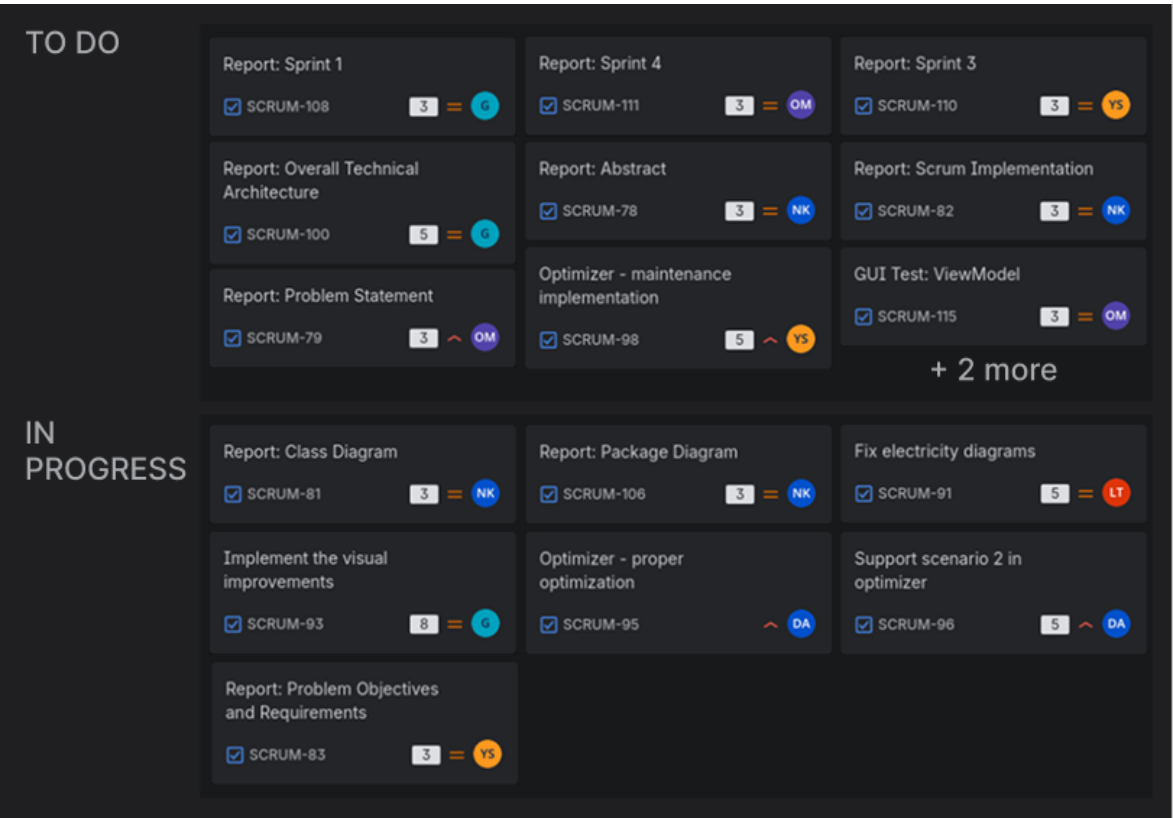


Figure 3.9: Overview of the Jira board at the start of Sprint 5

Sprint Review

The sprint ended with most of the planned tasks completed, showing solid progress from the team. However, several tickets got stuck in the review stage, which slowed down the overall workflow. In addition, code review and validation took longer than expected, creating a bottleneck that delayed the final integration of some features. Despite these challenges, the team delivered most of the intended functionality.

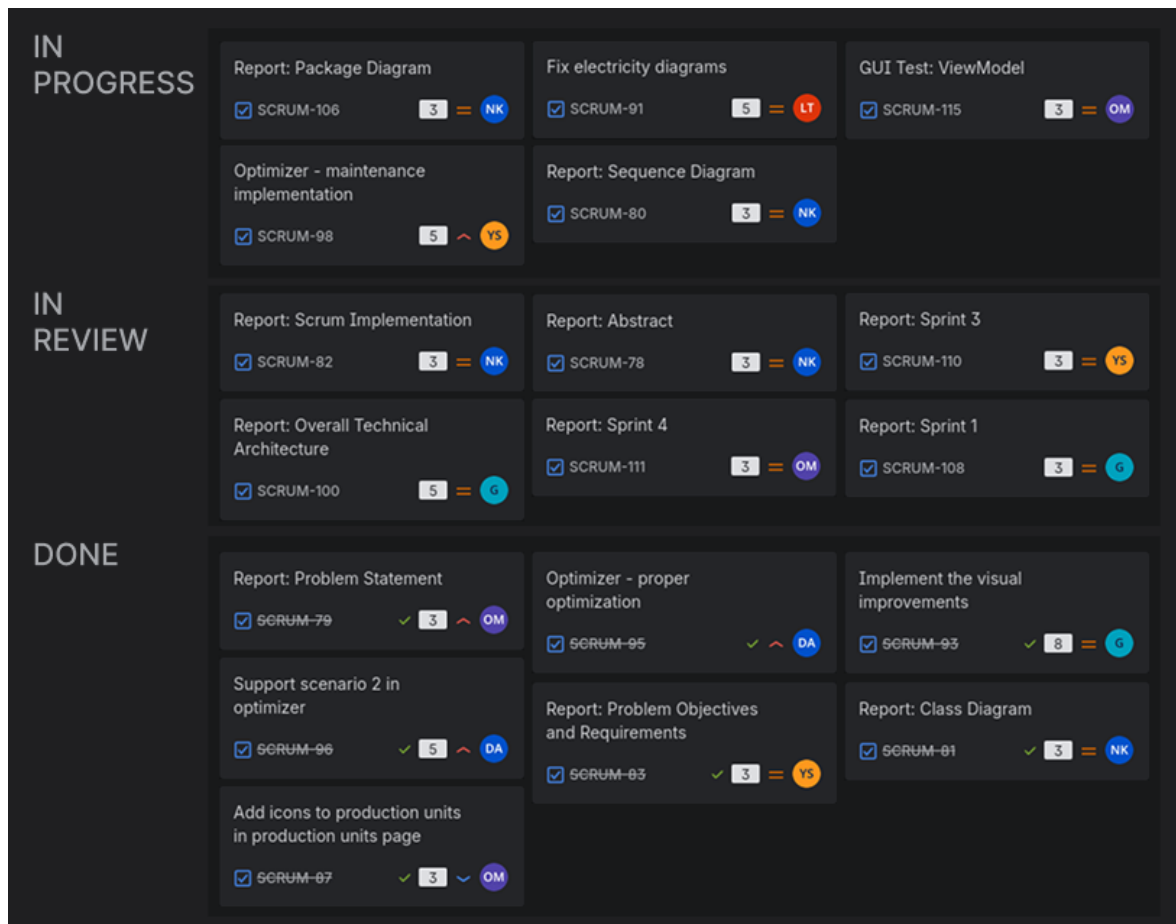


Figure 3.10: Overview of the Jira board at the end of Sprint 5

Sprint Retrospective

Communication about the tasks seemed to have improved and the progression was going well. The members have started to give more elaborate answers to questions and are voicing their opinions more. Lastly, the workload was adjusted to the scheduled mathematics exam.

The sprint progress was slower due to external factors such as the mathematics exam, a slightly reduced availability due to homework projects and other communal issues. Additionally, absences from meetings influenced progress. Notifications of absence were often sent very late, which caused organizational issues. Meetings were not always rescheduled when key participants could not attend. The group contract, which requires attendance and at least one-day advance notice of absence, was not consistently respected or enforced. Half of the team members did not meet the estimated workload defined during sprint planning by a significant margin. A large number of tasks accumulated in review and were repeatedly moved between “In Review” and “In

Progress,” causing delays. This issue was further worsened by slow review activity and insufficient follow-up. As a result some tickets from the previous sprint remained in the review cycle and were not completed during this sprint. Additionally, there were inconsistencies in expectations regarding hybrid meetings and short-notice changes, which further affected coordination and planning. Although team communication has improved overall, some members still occasionally forget to inform the group about absences or meeting arrangements, which contributed to coordination issues. During one of the meetings, attendance was split between online and on-site participants, which highlighted the lack of a consistent approach to hybrid meetings.

The attendance issue will be solved by requiring team members to send a message in the group one day before the meeting, confirming whether they can attend in person at a specified time and location or, if not, whether they can attend online. This approach accounts for the possibility of late notifications about online only attendance and allows the team to prepare for hybrid meetings in advance. If a significant number of team members are unable to attend, the meeting should be rescheduled. Regarding the review process, team members will be reminded that, as previously agreed, participation in code review is required from everyone to ensure consistent workflow and progress tracking.

Reflections and Conflicts

Communication and engagement improved, and the workload was adjusted for the mathematics exam. However, progress was slowed by attendance issues, late notifications, and tasks getting stuck in review or not being completed from the previous sprint. Coordination was also affected by inconsistent meeting arrangements and expectations. For the next sprint, we will enforce earlier attendance confirmation, improve planning for hybrid meetings, reschedule when necessary, and set clearer deadlines for reviews to improve flow and completion rate.

3.7 Sprint 6

Summary

The goal of the Sprint was to polish up the project and fully finish it. The code was refactored to accommodate for fixes, notably for the Optimizer in the Backend and the

diagrams in the Frontend, and implementation of additional Unit and GUI tests. The documentation was also finalized and reviewed.

Deliverables

The fixes were fully implemented which allowed the Optimizer to properly match electricity prices with units, have a proper maintenance selection, and make the diagrams more clear and detailed. The report was finished and properly reviewed to ensure it is to an utmost standard. The complete version of the report was exported to LaTeX in a separate GitHub repository from the project itself.

Sprint Planning

The Sprint goal was to fully complete the tasks which included fixing optimizer, production unit page, screen navigation panel and diagrams, GUI tests for ViewModels and an End to End (E2E) test, and adding sections to the report - Diagrams, Sprints, Implementation, Technical Architecture, Discussion and Conclusion. For each section added, other members would review it to ensure it is up to terms with the rest of the report. The team also decided to have daily work meetings where by the end Daily Stand-Up was held. Overall, the focus was to finish the remnants of Sprint 5, and the new tasks for which Planning Poker was used to determine story points.

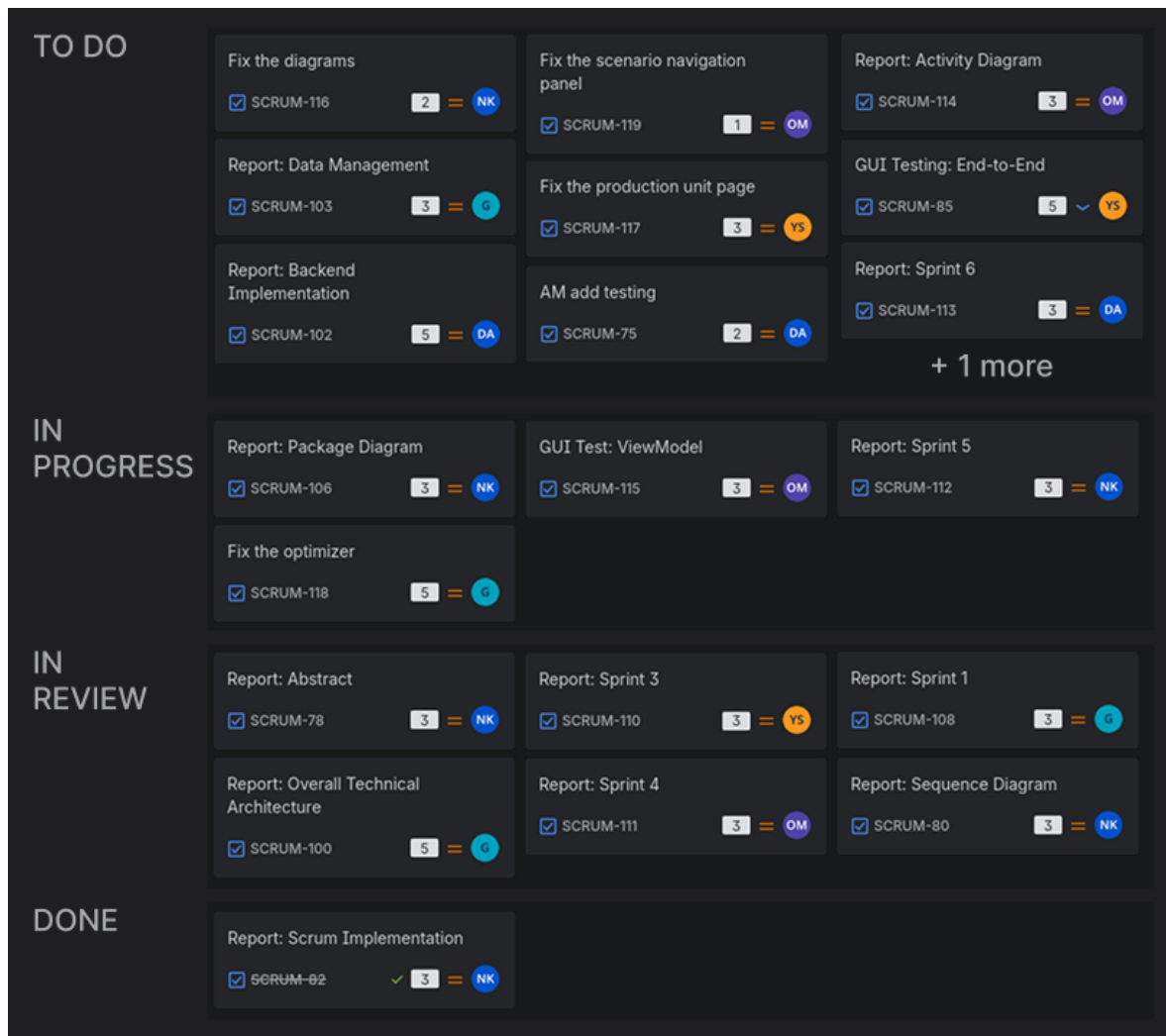


Figure 3.11: Overview of the Jira board at the start of Sprint 6

Sprint Review

The Sprint ended with all tasks being achieved, which has shown as significant growth from the team compared to previous Sprints. Story points however were not estimated properly as some tasks took longer than the given time which resulted in slower progress than usual. There were additional tasks added to the board that were not part of the start of the Sprint, and they were also finished. Despite this hurdle, we essentially finished all the tickets that were assigned and tasks that were left outside the Jira board due to their smaller size and significance.

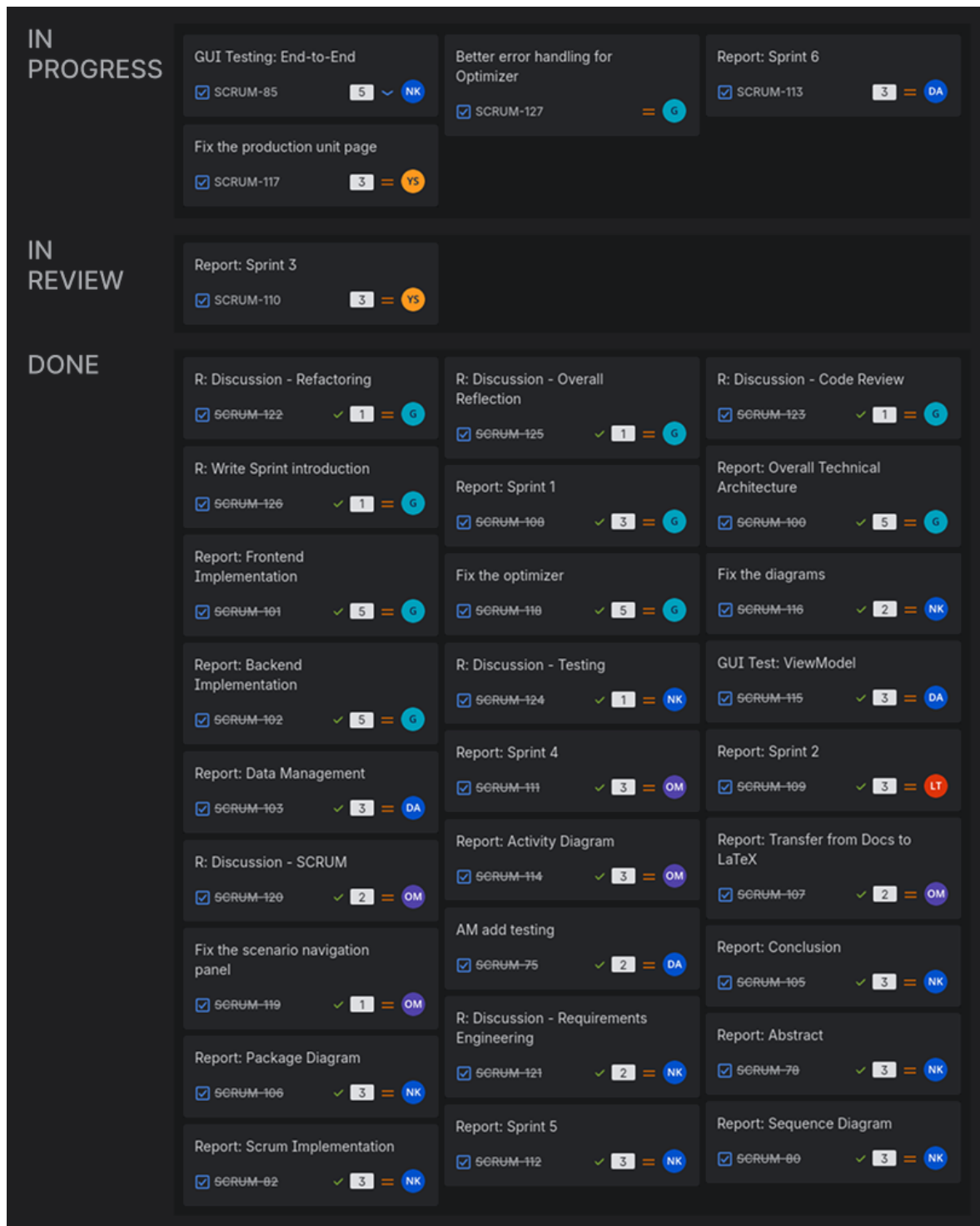


Figure 3.12: Overview of the Jira board at the end of Sprint 6

Sprint Retrospective

The use of the daily work meetings with daily Stand-Ups at the end of the meeting helped with the progression of the tasks along with the workflow. Communication regarding task distribution and progress was therefore improved which allowed for

greater collaboration between members. A hybrid model of meetings was conducted which was proven useful for the team. Reviews also improved as there was a noticeable increase of them, both for the project and report. The planning of the tasks as well as their description were well thought through and went into the team's favour. Even with the greater amount of tasks to do, many of them were completed along with sending feedback at the appropriate times. What also contributed to this Sprint was the lack of external priorities - lectures, exercises, homeworks, etc.

The major issue was the amount of tasks for the Sprint 6 which outnumbered most of the others. Some of the tasks themselves proved to be a challenge and took longer than the assigned time which slowed down progress. Although communication in terms of progress had improved, and the hybrid meetings were generally beneficial for the team, they still had drawbacks. Most notably, the downgrade of real-time engagement compared to physical meetings, which led to some inconsistencies between teammates and quieter meetings. Around the halfway mark of the Sprint, it was decided to migrate to a LaTeX version of the report as planned, but also keep the Google Docs version which led to a lot of confusion. A member of the team also used AI without disclosing it to the team which led to issues in the feedback. The issue was not discussed properly which was a breach of psychological safety. Some members would not notify the rest of the team about important matters such as a state of progress or being late for the meeting. Due to the quick progress of smaller tasks, reviews became more difficult to track which led to uncertainty as to what was reviewed and what is yet to be checked.

It was decided that important topics should be discussed to a greater extent, especially guidelines and rules. More members should participate in Pull Requests both for the report and the project. Better organisation for communication such as more channels in Discord, which include one for "Rules and Guidelines" and another for "Frequently Asked Questions(FAQ)", and better organisation for Excalidraw canvas such as clearer way of notifying of changes and a clean board where the rules are set.

Reflections and Conflicts

Communication was vastly improved due to the increased amounts of daily Stand-Ups even with the issue of inconsistencies with hybrid meetings. Since the team was free of external pressures, there was full focus on completing the project which benefited the progress during this sprint. Scheduling of the tasks was also improved as there were far less incorrect assumptions of timing when compared to previous sprints, although some still took more time than the initial estimation. Despite this, confusion and mistakes still arose especially when it came to the report with inconsistent reviews and pressure

for which version to edit. The issue with the undisclosed AI usage did affect the team's progress, but we managed to recover and address the issue. It was decided that sources of communications must be improved to avoid similar cases of happening which include better notification of changes, and an easier way of setting up the rules.

Chapter 4

Solution (Technical Aspect)

4.1 Package Diagram

The package diagram was included to show how the Heat Production Optimization system is structured into separate modules and how responsibilities are distributed across the application. It helps to understand the overall architecture by clearly separating data management, core logic, and visualization making the system easier to maintain and extend.

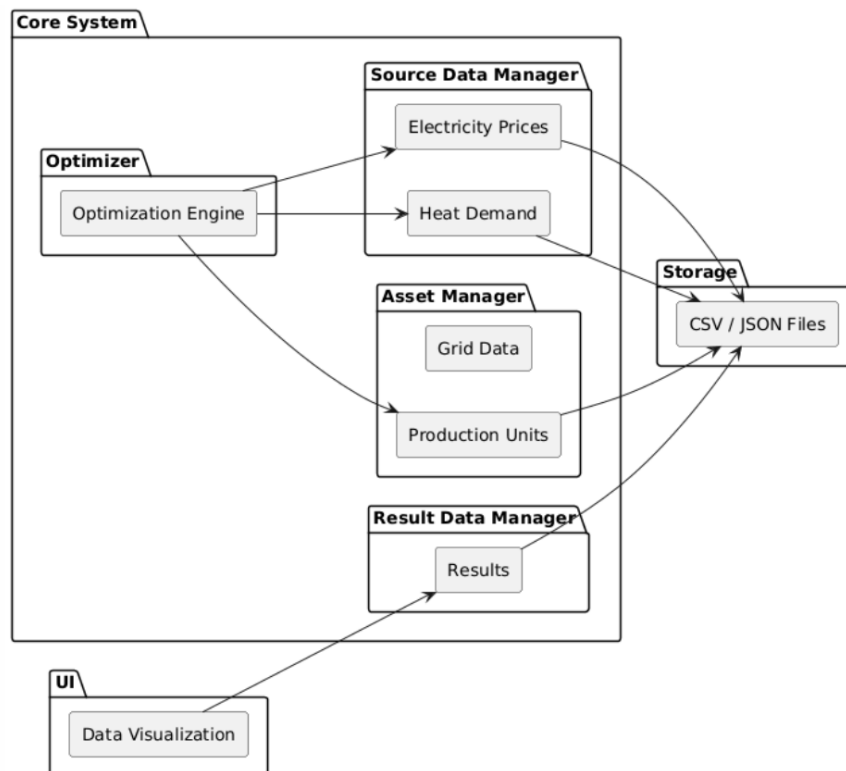


Figure 4.1: Package Diagram

The system consists of several key parts: the Asset Manager, the Source Data Manager, the Optimizer, the Result Data Manager, and the Data Visualization module. The Asset Manager takes care of keeping fixed information about the heating grid and production units, while the Source Data Manager deals with changing data such as heat demand and electricity prices. The Optimizer is the main part of the system. It takes in information from both managers to create the best schedule for producing heat in the most affordable way.

Once the calculations are finished, the results are saved in the Result Data Manager. Then, they are shown to the user in an easy-to-understand way using the Data Visualization module. All modules work together using a common storage system that relies on CSV and JSON files. This setup allows for the saving of data and the sharing of information between different parts. This package setup was selected because it clearly divides the tasks of managing data, optimizing processes, and showing information. This makes the system clearer, easier to maintain, and able to grow. It also helps with future development and adding new features.

4.2 Class Diagram

The class diagram was included to illustrate the overall architecture of the system and how the main components interact through a central DataManager. It helps to understand how data flows between modules and how responsibilities are distributed across the system, particularly in relation to data handling, optimization, and result presentation.

The diagram shows a system built around the Data Manager which acts as the main part connecting the Asset Manager (AM), Source Data Manager (SDM) and Result Data Manager (RDM). It organizes activities like bringing in data, improving it, choosing time periods and sharing the results.

The AM takes care of Asset objects and ScenarioData that hold details about capacities, costs, emissions and maintenance rules. The SDM manages SourceData objects that monitor data that changes over time like heat demand and electricity prices. The RDM keeps the ResultData produced by the Optimizer which includes the results of the optimization and a summary of the statistics.

The Optimizer takes information from the AM and SDM to help with planning production and figuring out costs. Meanwhile, classes like ScenarioLoader and JSONPersistence help with loading and saving data.

In general, the design divides tasks among different classes, which makes it simpler to keep the system running, add new features, and make updates.

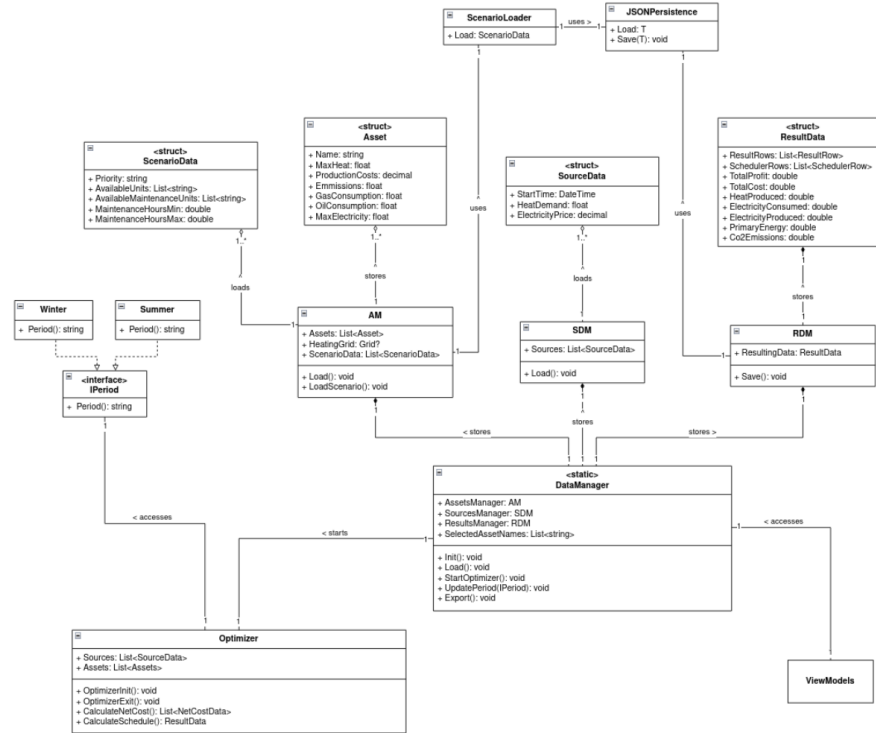


Figure 4.2: Class Diagram

4.3 Activity Diagram

The activity diagram represents the user interaction flow with the heat optimization system. The user begins with choosing the desired scenario followed by the heating period. Then, he or she can either proceed to edit production units or directly to the execution of the optimizer. After the system calculates the results, the user can export the data, view the charts or exit the program. The decision paths shown on the diagram represent the flexibility of the flow as the user can revisit earlier steps.

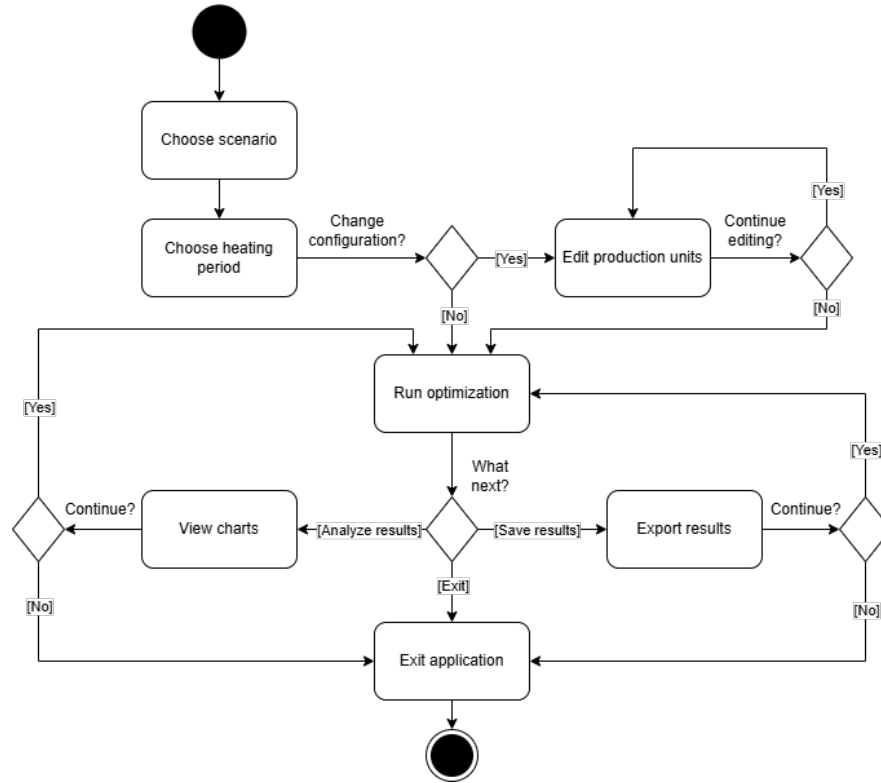


Figure 4.3: Activity Diagram

The diagram models the optimization execution process from the perspective of the user, rather than low-level implementation. Therefore, it provides a clear and intuitive representation of the workflow, which can be used to communicate the system behaviour to non-technical readers or stakeholders. Moreover, the diagram is used as a validation of the business objectives and high-level requirements, and to provide documentation for future maintenance.

Actions and the flow presented on the diagram connect to the implementation of UI functions and how user actions trigger system events. The support of loading pre-configured scenarios allows the user to run the optimization right after selecting the scenario and heating period, without the need to edit the units. The optimization and production units' configuration pages are separate, and they can be accessed at any time. Once the optimization is started by pressing the Run Optimization button, the charts are updated with new data, and the export button becomes enabled. The user can choose to either view both or one of them, or directly exit the program.

4.4 Sequence Diagram

The diagram was included to illustrate the system workflow and how data flows from user input through processing to result visualization. It helps to clarify how the View-Model layer, DataManager, and Optimizer interact during the full optimization process, from scenario setup to result presentation.

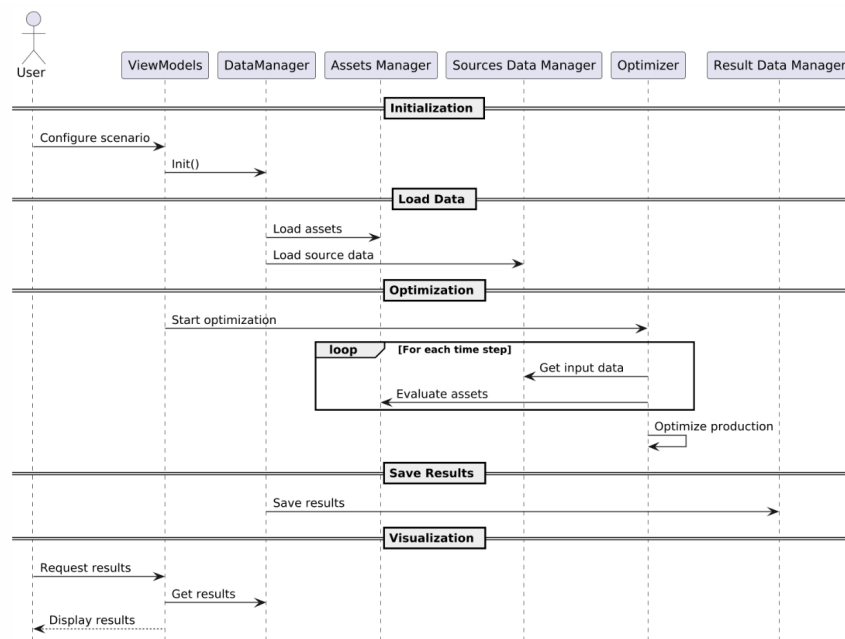


Figure 4.4: Sequence Diagram

The process starts when the user sets up a scenario and chooses assets using the View-Models. This triggers system initialization via the DataManager. The DataManager gets information about assets from the Asset Manager, and collects time-related data like demand and electricity prices from the Source Data Manager. After gathering all the necessary information, the user starts the optimization process and the data is sent to the Optimizer.

The Optimizer looks at the data step by step for each time period, checking the demand and price situations. For every asset, it figures out the production costs, ranks the assets according to how economically efficient they are, and decides the best way to allocate production while following the operational limits. Once the optimization is finished, the results are gathered and saved in the Result Data Manager.

When the user asks for it, the ViewModels fetch the results and show them in the user interface through charts and important performance indicators. This makes sure there is a clear division between calculating, managing data, and showing information.

This design was selected to keep a clear distinction between managing data, optimizing processes, and showing the user interface. It makes it easier to take care of, helps the system grow, and allows for effective management of complicated optimization results, all while keeping the system workflow organized and easy to understand.

4.5 Overall Technical Architecture

Technology Stack

The technology stack for second semester project was C# for all frontend and backend, C# Avalonia framework [6] for frontend with data bindings, LiveCharts [7] for data visualization, DialogHost.Avalonia library [8] for dialog boxes, and NUnit [9] for testing. C# allowed to easily develop a desktop application for multiple platforms, integrate with external libraries, and all team members had experience with C# prior to the project which helped with code contribution early on. C# Avalonia framework allowed to visualize the data, and interact with the system easily. LiveCharts library for Avalonia enabled the display of time-series graphs. It further improved readability of user interface and the data displayed. NUnit testing library was used to create and run unit tests. Usage has improved the project's code quality and robustness, and helped to prevent regressions.

Technology Stack Challenges

Working with Avalonia proved to be more challenging than expected as online resources, documentation were not as high-quality and satisfactory as expected. Provided abstractions by Avalonia and their suggested design patterns did not allow for scaling code well while keeping codebase clean - data duplication, minor hacks for reactivity and passing data were required to achieve desired result and many of their official examples introduced substantial complexity. Additionally, LiveCharts library was limiting because it had issues with crashing outside of C# code. For loading, storing and exporting state, JSON files were used. Due to small project scope, low volume of data, high flexibility, easier data management and the project being a standalone desktop application, storing data in JSON was more fit for the task than using traditional relational databases such as PostgreSQL [10] or MySQL [11]. The team has considered using Docker to standardize the development environment even further but the idea was rejected because there were no major issues of the application not

working as expected on a different computer. There was only one issue during Sprint 1 related to different path handling on Windows and Linux but it was fixed quickly. Furthermore, team members have considered integrating CI to GitHub [12] for running tests automatically, but it was ruled out as unnecessary because during code reviews reviewers already run tests among other review tasks and there would be no increase in development velocity from such integration at such project size.

Seperation of Concerns

To keep the codebase clean, make it modular, easier to work with, and collaborate, 3-layer programming was used - application was split to presentation, domain and data layers. Additionally, the application was segmented into five modules. Asset Manager (AM) responsible for loading static data, Source Data Manager (SDM) responsible for loading dynamic data, Result Data Manager (RDM) for storing result data and exporting. These modules belong to the data layer. Optimizer (OPT) responsible for considering all available data, scheduling available units optimally to reduce costs and scheduling units for maintenance, and Data Manager (DM) which connects the data layer to be accessible to the Optimizer module and the presentation layer. Data Manager acts as a thin wrapper around the data layer for the Data Visualization module to access it and for Optimizer it provides wrapper methods to call. These two modules belong to the domain layer. The Data Visualization (DV) module forms the entire part of the presentation layer. It is the single largest module and it is further broken down to different views and respective view models. Different tab groups are separate views such as vertical scenario tab group and upper page tab group. Each different page tab is a different view, such as overview, optimizer and production units page. Smaller elements such as different graphs and edit production unit popup have their own views. Splitting data visualization to many different views proved helpful for readability, reuse and reducing merge conflicts.

Architecture and Rationale

3-layer programming architectural pattern was implemented as the foundation for the codebase. In the Data Visualization module, each view had their respective view model with data bindings and commands, and it followed MVVM design pattern. The decision was made to choose MVVM over other GUI architectural patterns because it increases readability, reduces boilerplate, and reduces code complexity as long as only simpler Avalonia features were used. To keep GUI reactive to data changes, the observer

pattern was used. Whenever data updates, a message is sent notifying that refresh is required and GUI components subscribe to the channel. Observer pattern looked like a perfect candidate for the task because it is heavily used in GUI development, is tried and tested, and keeps single responsibility principle with no significant downsides. However, due to the difficulty of passing events to components, only some parts of the code use observer pattern. Data Manager module which connects different modules acts as a singleton by using C# static members.

The code follows DRY principle of not repeating code by using modules and utility classes. For example, JSON loader functionality is used in importing and exporting data. Instead of copying over the functions, a new JSONPersistence class (refer to SE2.Data/Persistence/JSONPersistence.cs) was created which handles all JSON related interactions. The benefit of improved readability was a decisive choice. However, some Avalonia components were proven to be difficult to cleanly refactor to proper modular components without adding significant complexity. Furthermore, the codebase follows YAGNI principle of not having unnecessary code which is unused. For instance, the project could have used an external relational database but not to overengineer a basic project, simple file-based storage approach was chosen. Quite a few parts of the codebase were refactored and removed during the development which were not needed anymore or a better, more simple approach was found. SOLID principles were partially adhered to. Single responsibility is used heavily by splitting many features into different modules and classes. For Object-Oriented Programming class design, composition over inheritance was heavily used. Various edge cases of inheritance such as diamond problem and potential complex situations later down the line deterred from using inheritance. Composition had no drawbacks in this case. None of the classes besides the Data Visualization module use inheritance which only inherit Avalonia base classes to implement functionality. As such, Liskov substitution principle is upheld. Open-closed principle is partially followed, as due to heavy use of composition, changing related classes can extend the behavior of the classes without modification but there are no specific mechanisms beyond composition to allow extending class functionality without modification. Interface segregation principle and dependency inversion principle were not strictly adhered to because there is only one interface in the codebase. Instead, composition on concrete classes were used. Decision was made to follow KISS principle to keep simplicity instead of introducing additional abstraction layers using interfaces. As an example, Optimizer (refer to SE2.Domain/Optimizer.cs) could have an IOptimizer interface but there is only one optimizer implementation and there would be no benefit in having multiple optimizers in our project - requirement is to only to optimize for best cost. Having an interface for Optimizer would add additional boilerplate code without any benefit. In such a relatively small project, additional simplicity and

reduced boilerplate allowed for quicker iteration.

Chapter 5

Implementation and Technical Realization

This chapter explains how the system was technically designed, implemented, and improved during the project. Its purpose is not only to show what was built, but also how the group worked technically and why specific engineering choices were made.

5.1 Frontend Implementation

Frontend for this project was implemented in C# with Avalonia UI framework, LiveCharts for graph visualization, and DialogHost.Avalonia library for dialog boxes. MVVM pattern and data bindings were used with Avalonia. Frontend is split into supporting elements and pages.

Supporting elements include sidebar navigation panel, top navigation tab bar and chart component. The sidebar navigation panel has the logo of the project and a list of scenarios which can be switched by clicking on buttons. The top navigation tab bar contains tab names for each page and a combobox for selecting different periods. The period checkbox is separate for each scenario. The chart component is used for dynamically adding charts to the page. It is used in the optimizer page.

There are three pages - overview, optimizer and production units page (see Figure 5.1). After the initial Figma prototype, there were four pages, the fourth one being the configuration page which handled scenario specific data. However, later down the line the production units page and configuration page was merged into one production units page because it improved user experience, clarity and reduced clutter. The first page,

overview page, contains a map of the Heatington city and various general graphs such as heat demand and production, electricity prices which are populated when the optimizer is run. The second page, optimizer page, has a button to start the optimizer and to export the results of the optimization. Furthermore, after the optimization completion, it shows totals for profit, expenses, amount of heat produced and consumed, energy used and amount of CO2 emissions. Maintenance periods are shown. The graphs with detailed optimizer information such as unit scheduling, price for 1 MWh for each unit at a given hour and costs for each unit at the given hour are shown. It is possible to use a combobox for selecting specific units to be shown in the graphs. In the third production units page, all of the production units are shown with their properties - image, name, heat production, electricity, minimum and maximum maintenance hours, if the unit can be maintained, unit colour in the graphs and so on. In the same page, production units can be selected or deselected for the scenario, units can be edited, deleted or new production units can be added.

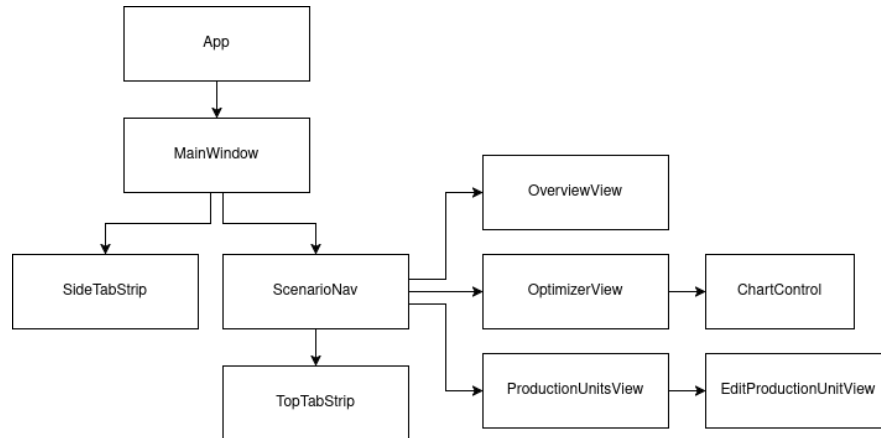


Figure 5.1: Frontend Diagram

Each component and page have their respective view models responsible for filling the data required for the component. All of the data and actions performed go through Data Manager directly or indirectly. Much of the data displayed in the UI is displayed directly without any modification and a small part of data shown is heavily processed data with inputs coming from Data Manager. Such data include data provided to the charts which contains numerous LINQ queries, handling unit selection, dynamic colors and empty data periods.

During the frontend implementation, major challenges were faced with writing clean code with Avalonia UI framework. First of all, data which is stored outside of Avalonia is difficult to update reactively without desynchronization issues. For example, Data Manager provides result data for the optimization and the result is stored outside of Avalonia code because it is part of the domain layer. Avalonia does not update the UI

with new data automatically. Instead, C# events could be passed around through the entire hierarchy of UI components to the Optimizer view model to call a refresh method once new data is available. However, such “prop drilling” to pass a refresh event was proven difficult as getting data to the view model from other components would have required changing every view and view model along the UI hierarchy. Unfortunately, some less efficient reactivity methods needed to be used, such as calling refresh manually from the view model itself when it thinks new data should arrive. For instance, after calling the run optimizer method, a refresh method is called for the Optimizer view model to populate fresh data. Such a method helped to solve the issue but due to its nature of not being so reactive to changes, many bugs have arisen where data was stale and caused desync. Adding more points to refresh data helped a bit more. In some cases, Data Manager was used to store data meant to be accessed between multiple components - such as current scenario and period. It worked but it caused additional issues with multiple scenarios and periods, and required granular tweaks to make it be more reliable. In hindsight, paying the cost of changing many UI classes to pass around data to components could have been better and more reliable than relying more on Data Manager for passing data around between components. Furthermore, a clear and consistent way to pass data around Avalonia components from the project start could have resulted in better results. Even multiple attempts to refactor code were not able to resolve major issues of the codebase.

In addition to Avalonia C# framework, LiveCharts library had its own issues too. After adding a few graphs with a total of around three thousand points, there were visible performance issues when resizing the application window. Because view model data did not change during application window resizing, it could have only been caused by the LiveCharts library. In the Optimizer page where there were more graphs than could fit in the screen, charts which were rendered off screen were hidden or distorted even when they scrolled into view. The hover tooltips would still show up but the visible chart had issues. The only workaround found was resizing the window to refresh LiveCharts charts views. Furthermore, a LiveCharts feature of adding disjointed line sections was used. For example, when a unit is running, then is powered off and turned on again, the points should not be connected and the gap should be shown instead. However, the lines at the ends of intervals in the charts had higher values and caused confusion about what values are really at those endpoints. But the points were rendered correctly and the hover tooltip was showing correct values which could only mean there is a bug in the LiveCharts library itself.

Frontend was designed to be a balance between being user-friendly and having in-depth features. It caters to simple users by being graphically appealing, having simple summary statistics and summary graphs, limiting the number of pages visible and

providing an intuitive interface. For advanced users, a feature was added to create, edit or delete production units and to change scenario data dynamically. Furthermore, they can inspect the optimizer better with the help of detailed scenario graphs which show granular key metrics such as unit cost for 1 MWh, unit cost for unit and scheduling of each unit. The team carried out two major Figma design iterations over the course of the project and a few of the changes were made directly in code after feedback from the stakeholders. Sprint reviews helped with improving the frontend incrementally by collecting stakeholder feedback at regular intervals and changing user interface based on the feedback.

5.2 Backend Implementation

Backend is composed from the Optimizer and Data Manager. Optimizer is responsible for calculating optimal cost for the provided time period and assets and Data Manager is responsible for exposing data layer to domain and presentation layers.

Optimizer is a central part of the backend. Data Manager configures optimizer by loading sources, assets, and any scenario specific data. During Optimizer initialization, data sources, and assets are validated. After validation, net cost for each unit at each hour is calculated. Net cost is the cost of 1 MWh of produced heat for a given unit at a specific hour. Net cost calculation takes into account production costs of the unit and any electricity produced or consumed by the unit during heat production. After initialization, the optimizer is started in a separate thread. Optimizer tries to find an optimal maintenance period for one production unit from the list of available units to be maintained. Optimizer iterates over all possible maintenance periods for each unit and selects the maintenance period with the lowest cost. For each possible maintenance period, the Optimizer step is run which calculates the cost, heat production, electricity consumption among other properties with the given maintenance period. During this step, maintenance period for the unit is applied and for each hour the units with the lowest cost are selected until the heat demand for an hour is filled. After heat demand for every hour is fulfilled, the result with granular detailed data and total summaries are returned back to the maintenance period iterator. The lowest cost maintenance period with its result data is stored in RDM. The main thread is notified about the completion of the optimization.

Currently, Data Manager belongs in the domain layer and it is responsible for exposing AM, SDM, and RDM to optimizer and frontend. Data Manager does this by providing a thin wrapper over AM, SDM, RDM and by providing methods which abstract

interacting with the optimizer and updating scenarios and periods. Data Manager implements singleton pattern using C# static members because there is only one globally accessible instance of Data Manager.

5.3 Data Management

The project uses a structured data management system that handles loading, storage, processing and exporting data. The data itself is modelled through using dedicated struct or data classes, including Asset, Grid, ScenarioData, SourceData, and ResultData, which define the information used throughout the backend. The system separates the static and dynamic data which improves flexibility, scalability, maintainability, and consistency.

The AM helps to load the fixed data both for the heating grid and the production units. It uses a JSON file deserializer structure which makes the code more flexible and scalable. JSON was chosen because it is easier to structure and maintain for complex data compared to CSV files. To load the file, it tries to find the path to the JSON file and from there, by using a StreamReader, the AM gets the information from the file. After that, it deserializes the file as the struct classes Asset and Grid and if the assets themselves are empty, it initializes an empty list to preserve consistency and avoid null errors. It also loads the ScenarioData with the help of the ScenarioLoader. Moreover, the AM has the method GetAssetByName which goes through them as a list and returns them as names only, and helps the backend systems load them properly.

The SDM is the manager that loads the dynamic data. When it reads the file it splits by row, since it uses a CSV file structure rather than a JSON one like the others. After reading through the file the SDM divides the sections based on the SourceData file structure in parts(array) - StartTime, HeatDemand and ElectricityPrices - the latter two of which contain CultureInfo.InvariantCulture to ensure consistent formatting and parsing of values. After that it adds each set of data. To differentiate between which time period is the data set in it, the SDM determines the path to the CSV files and with the usage of an interface called IPeriod which contains two classes inside it, Summer and Winter, each of which helps to direct to the exact file. This improves scalability because it can be added on top of the existent code without changing the logic.

The RDM is the saving and exporting mechanism of the project. Just like the AM, it uses JSON files because they are more flexible and scalable compared to the CSV ones. JSON also allows the result data to remain structured and easy to process.

For the saving mechanism, the RDM uses a dictionary list where the key is used to return the set scenario result data and if there is not a key then it returns null. It ensures scalability and consistency between scenarios and their respective results. This scenario is then saved by using IPeriod to determine from which period the data is from by checking the path to the file and determining the exact file of choice.

The ScenarioLoader helps the AM to load each scenario by helping to find the path by differentiating the scenario number. To achieve this, it uses JSONPersistence for help. The ScenarioLoader is used as it allows for an easier and more scalable way of loading each scenario. This improves scalability since more scenarios can be added without refactoring the code. It also separates scenario-loading responsibilities from the AM thus improving maintainability.

The JSONPersistence class allows to easily implement the load and save mechanism for the JSON files, which not only simplifies the code that appeals to the KISS principle but also the DRY principle to not have to duplicate code multiple times. It performs validation checks to verify the existence of the file and the success of the deserialization, which ensures the consistent handling of the JSON data.

Data and Struct Classes are the files that help the managers load and save the information, such as Asset and Grid for the AM. They are used as templates for how the output of the managers should be formed which improves consistency and management. They also define the overall data structure throughout the project. This is why the decision to use them was made.

Chapter 6

Discussion & Reflection

6.1 Scrum

The use of the Scrum ceremonies, artifacts and the core values positively influenced many areas of the project. Compared to the previous semester project, there was more consistent progress, better structure, clearer responsibilities, and higher understanding of the project among all members. Scrum especially improved accountability, team dynamics, workflow, and even code quality. The improvements were progressively more noticeable from sprint to sprint, due to the Scrum's iterative nature and learning loops.

Accountability

The implementation of Scrum has significantly improved accountability within the team. During Daily Stand-ups, the members were faced with the obligation to give an account of their work, and no one wanted to attend without any progress as they did not want to disappoint others. Especially at the beginning, it was mainly about fearing negative judgement, but with time and improved psychological safety, the team members wanted to contribute more based on collective responsibility. Moreover, Sprint Review enforced the responsibility among the members so that the team had a significant increment to present in front of the stakeholders. The additional estimate of everyone's own work time, which was added to the team's Sprint Planning, further improved the accountability as everyone evaluated their own capability and no one wants to contradict their own words. In general, the transparency that is central to Scrum helped with greater awareness of all members' work, leading to people feeling

more accountable for the whole project.

Team Dynamics

Over the time of the project, Scrum led to better team dynamics. Daily Stand-ups and transparency helped the team be more motivated, as they saw everyone else working on the project. Due to frequent meetings the members became closer and more comfortable with each other, which improved the psychological safety in our team. It further influenced the extent of transparency and effectiveness of resolving issues, which was seen especially during Sprint Retrospectives. As psychological safety increased, members became more open, started to voice their opinions more and it was easier to bring issues up. Compared to the previous semester project where issues were avoided, here they were addressed more effectively each sprint. Not only were issues more often discussed, the team came up with solutions together. Even if not all solutions were successfully implemented, many issues were resolved. Moreover, the collective decision-making, present at Sprint Planning and Sprint Retrospective, also improved psychological safety, as team members felt their input was valued. The integration of the Excalidraw board was especially important for letting everyone contribute their opinions. In turn, it encouraged greater participation in discussion, which further strengthened the collective decision-making process and communication in general.

Workflow

The Scrum framework was efficient in preventing procrastination and staying on track with the tasks. Since Sprints are timeboxed, progress on the project stayed somewhat consistent throughout the whole duration. Timeboxing helped with not leaving the work for the last moment possible and managing scope creep as tasks were broken down. Moreover, setting a sprint goal during Sprint Planning supported the team to keep the direction and avoid sidetracking. Daily Stand-ups provided an insight into the progress making it more manageable to control the team's flow, especially when tasks were seemingly stuck in progress as the members could resolve issues faster.

Code Quality

Even though Scrum does not define coding standards or practices, its implementation indirectly influences code quality as it improves structure, transparency and account-

ability. Since the work on the product is split into many sprints, the tasks were much smaller which made code reviews easier and quicker, allowing for more frequent reviewing. This made the team members more conscious of the code they were delivering, as they knew it would be evaluated. Moreover, because of high transparency and Daily Stand-ups, everyone had a better knowledge of the whole project and therefore a more well-structured and coherent code was produced. Sprint Reviews also contributed to improved code quality as frequent stakeholder feedback worked as quality check and it helped with staying aligned with the requirements.

Challenges

Roles

The team's implementation of Scrum roles was not the greatest. Despite the project simplifying the roles essentially to Scrum Master and Product Owner, they were not well established from the start which led to issues. The Product Owner was assigned at the beginning to one member; however, it was not clear what were the responsibilities and the existence of the role was quickly forgotten. It is possible that it was not easy to maintain the role due to lack of contact with the external stakeholders. The Scrum Master's role was also not clearly defined at the beginning, leading to problems after the decision to change the assigned person every Sprint. The problems included inconsistency in the organization of meetings and Sprints in general, sometimes negatively affecting the progress. However, alternating the Scrum Master was deemed beneficial for individual learning and understanding of the role, therefore around the middle of the project the role was provided with more descriptions of responsibilities. In future semester projects, the responsibilities of each role will be defined to avoid ambiguities and prevent disorganization.

Planning and Estimation

The Sprint Planning ceremony was not as helpful nor as expected, mainly due to poor time allocation. The ceremony was scheduled for Mondays, when the time block was sometimes occupied by lectures or was simply not enough to finish the planning. The decision to move it to a later hour was not approved, as lectures were finishing late, however moving it to Tuesday did not fit either because it would shorten our Sprints. Therefore, planning was often rushed and in return tasks were not well described in the Product or Sprint Backlog as there was not much time for discussion. This was a

significant issue at the beginning of the project as the team members did not yet fully grasp the requirements and details of the program, which led to impediments during implementation. Moreover, despite assigning story points by playing Planning Poker, those estimations were not as useful. Tasks took longer than estimated even when the team was overestimating the story points. It was likely due to the poor time allocation of the members; however, it could also be influenced by lack of time to go in depth about each task during Planning Poker. From this experience, the group learned that Sprint Planning is essential to smooth progression. In the future, timing issues should be taken into account when scheduling the Sprint Planning, and the ceremony will be set during a time period where nothing else overlaps with it. This will to some extent prevent non-descriptive tasks entering the Sprint Backlog and therefore reduce impediments related to lack of clarity.

Communication and Engagement

At the beginning of the project, the communication was not satisfactory. At Daily Stand-Ups, members were giving accounts of ambiguous progress, briefly stating that they are working on a task, without any information about what stage they are on or of any impediments. However, with increased psychological safety, it became less problematic. Another issue raised closer to the end of the project, as meeting attendance started dropping. Perhaps due to some members finding meetings not as productive as they wished or losing interest in the long progress. Attempts to fix the attendance were made, such as changing meeting time and proposing hybrid meetings, that are both in-person and online, but it did not yield much improvement. For the next projects, message-based meetings will be proposed, so that all team members can participate in the meetings in their own time. The main format of in-person meetings will be kept, as it significantly improves workflow and accountability, but now members will be additionally encouraged to update their progress even outside of the meeting time. This will help in situation when some members are not able to attend meeting or do not feel like the meetings are contributing enough to their own workflow. Moreover, the meetings will be more structured and a timer will be set, in an attempt to prevent the team to have more focused and productive meetings.

6.2 Requirements Engineering

The methods the team used for requirements engineering had various effects on the quality of our project. Some were very important during the development process,

while others were mostly helpful in the early design stage.

At the start, functional and non-functional requirements (see Appendix E) were very important because they defined what the system would do, its goals, and helped to create the list of tasks to complete. As development went forward, the process became easier to adjust with new tasks coming up mainly from testing, feedback, and making changes to improve the code.

User stories gave a viewpoint which focused on the users of the system. At first these were part of the course requirements but later on they turned out to be helpful for explaining how users work, what they expect and how they interact with the system. This helped in creating features that are easy to use.

Use case diagrams show a general view of how actors interact with the system. They showed how different users interact with the system's features and explained the connections with other systems.

High-level requirements served as a middle step that connects broad ideas about the system to more specific functional and non-functional requirements. They were simpler to grasp and offered a clear way to move towards more detailed descriptions.

Activity diagrams were helpful for showing how users interact with the system in a clear, step-by-step format. They gave a straightforward summary of how the system works and helped with the initial design choices.

Sequence diagrams give a clearer picture of how systems interact, highlighting the complete communication between different parts of the system and outside systems. They helped make it clear how information and responsibilities worked within the system.

The UML class diagram had the biggest influence on the project. It acted as the main plan for the building, outlining how the system is organized, the different classes, their duties, and how they are connected. It helped break down the system and share tasks among the team, which made working together better. Even though the class diagram started off to help with the implementation, it slowly started to differ from the code because of feedback, bug fixes and changes to improve the code. So, the UML models were changed to show the new way things were being implemented instead of just telling how to do it.

Other methods, like package diagrams, CRC cards, and verb-noun analysis, were mostly used as helpful tools in the early design stage. Their real-world effect was not as significant when compared to the primary UML artifacts.

For future projects, it is a good idea to keep track of both functional and non-functional requirements, user stories, activity diagrams and class diagrams. These practices help to clearly define what needs to be done and what the goals are, keep the focus on the user, allow for an early look at how the system will work, and create a solid base for the implementation.

6.3 Refactoring

Refactoring was implemented as part of other tasks. Often, tasks in Jira which included new features or fixes required changes to existing components, thus required refactoring to complete the task. Refactoring helped to remove technical debt, and change the design to accommodate unforeseen architectural and feature changes. For example, implementing multiple scenarios support required refactoring a large part of the codebase. It helped to make the codebase a bit more consistent and cohesive. Larger refactoring was carried out by few people in the group for the most part who had more experience with programming. There was a skill imbalance in refactoring code because more complex refactoring required deeper understanding of the impact of changes and careful deliberation between different architectural approaches. More isolated refactoring for small components were carried out by many team members, often in the form of smaller fixes which were necessary to complete a broader task assigned to them. Those individual tasks were not strictly labelled as refactoring, rather as fixes or new features.

6.4 Code Review

Code reviews were implemented by using Pull Requests (abbreviated as PR) in GitHub for every single code change. Every single pull request had to be reviewed by another person, approved, and then merged into the main branch. Direct commits to the main branch were not allowed. For developing the project, the team has opened over 40 pull requests [13] in GitHub for code review, and some of them went through multiple revisions before being merged [14]. Code reviews helped us with knowledge sharing significantly. The more knowledgeable person would explain where the issue is, spot the potential improvements, and the other one would be able to learn and improve faster. The additional transparency to what others were working on and requiring a review for it led to better overall project understanding for team members. Furthermore, code reviews helped to make the codebase more consistent by asking changes to be

made when code deviates too much from expected standard. Additional checks by another person added an additional code quality validation layer which increased our code quality but it was not a silver bullet.

There were many challenges we faced with code reviews. One of the challenges was the time it would take from creating a PR to when a PR review would be completed. We overcame the issue by assigning a specific person to do the review for a specific PR. Another issue we faced was which reviewer should be assigned for a PR. At the start, only a few team members more experienced with programming were doing code review so it was more akin to a role rather than a task. Later down the line, we assigned a person to do a PR based who was more willing to review the pull request regardless of experience level, and who was already available for code review. The last problem we experienced were merge conflicts. They were rare because early on in the project we split lots of functionality in multiple files, kept PRs relatively small and bundled tasks together which change similar files. Any merge conflicts which appeared we have fixed manually fairly quickly.

6.5 Testing

Testing was significant for making the product better. It helped product quality by helping find bugs and incorrect behavior early. The project included manual testing, unit testing, GUI testing and end-to-end testing. The testing was helpful for confirming that smaller components of the system worked correctly and for checking edge cases that could be easily overlooked during regular use. The team ran the tests themselves while working on the project and before combining changes. Unit testing was done using NUnit and GUI testing used Headless Avalonia to test the user interface without requiring a graphical environment. For example, unit tests were used to check that the system created the least expensive heating schedule while still satisfying the heating needs. Tests also looked for any problems that came up after changes were made to the optimization logic and made sure that the production units worked correctly during the optimization calculations. On the other hand, manual testing made sure that the app worked correctly from both the user and the business point. GUI and manual testing were used to make sure that users could set up production units accurately and that the optimization results were shown correctly in the application. Together these testing methods improved the quality of the product.

Even with these advantages, testing also brought some disadvantages. Creating and keeping tests took a lot of time and hard work, mostly when changes to features led to

existing tests failing which meant they needed to be updated. For example, when the optimization features were changed many of the unit tests that were already in place started to fail. This meant that those tests needed to be rewritten to align with the new behavior. Often the tests mainly looked at separate and simple situations, which meant that big processes were not always completely carried. Moreover, since testing methods were put into place later in the development process the overall content throughout the system was uneven. This had happened because testing practices were only introduced during AOP around the middle of the project. In result some features were tested a lot more than others. Moreover, the team has written some integration tests to test the entire flow of the application. However, testing dialog boxes, which appear when optimizer is ran and when production units is edited, proved too difficult. The DialogHost library we are using for popups windows has apparently no functionality to access the content in the popups from the testing code. It greatly limited the scope of integration tests the team was able to write.

For upcoming projects, the team would include testing from the very start of the development process. Pull requests should only be approved if they have the correct test content and the system must be designed to make testing easy. It's achievable by using interfaces, designing in a modular way and using mock classes. This would help to maintain the code more easily and give people more confidence to make changes in the future.

6.6 Overall Group Reflection

Overall, the project managed to solve the issue. Heat production optimizer schedules production units optimally. All the requirements provided by stakeholders were satisfied. Only functional requirement 8 (FR08, see Appendix E) which was defined internally was changed. Functional requirement states that the system shall store optimization results in local CSV files. However, the team has decided to store optimization results in JSON files instead. The original stakeholder requirement of using CSV was a recommendation, then the team made it a requirement in the early stages of the project. Later on, after discussion internally in the team, and with the supervisor and a professor, CSV format was switched to JSON.

Reflecting back on the product, there are some unforeseen problems the application could cause. If the application were to be deployed in the real world, the reliability of the product would not be good enough for production use. Application errors would lead to downtime. Furthermore, the codebase contains technical debt which

accumulated from imperfect design, time pressure and lack of experience. Thus future feature development would be slower and parts of the codebase would need to be even more refactored. Issues with the Avalonia UI framework would cause major issues removing technical debt. Additionally, new requirements such as new different user personas using the product or new requirements for the optimizer such as taking into account CO2 emission would cause unforeseen codebase integration problems. The project does not scale well to the scale of production deployment - it would not be able to handle much data, significantly more assets and more sources because currently data is file-based and not stored in a database which is common in production deployments.

With the future work, these issues could be mitigated by migrating the application to production-grade architecture which would support large amounts of users and big volumes of data. The application would be rewritten to support distributed cloud architecture - relational database for handling big volumes of data, microservices for handling large amounts of users and faster feature development. New architecture would increase codebase scalability considerably. Adding more checks and validation for external data like resiliency to unstable external data sources would improve reliability. New potential features which could provide business value for stakeholders would be adding scenario navigator to navigate, create and delete many scenarios. Currently, the heating grid map is static and providing a dynamic interactable map could be more helpful. Ability to change images of the production units in GUI would allow more flexibility without leaving the application. To improve performance, the optimizer could run on multiple cores. New distributed architecture would be able to support a more diverse set of users. Adding authentication and authorization would allow different users in the organization to share their configurations and modify data live. Visual export feature could be provided with selective export options, which would generate visual representation and allow sending data to stakeholders which do not have access to the application. Multiple language support such as adding Danish support would help for users which are not as proficient in English. Eventually, the application would grow to be much more complex and adding AI summary or AI chatbot support would help with beginner users starting to use the application and help with getting tasks done quickly.

Chapter 7

Conclusion

In conclusion, our project was able to build a heat production improvement system with the capability to automatically schedule production units for utility company HeatItOn. The system allowed for low operational costs while producing heat and satisfying customer demand which resulted in a rise in overall profits. It considered electricity price, seasonal heat demand fluctuations, maintenance downtimes and production unit operating capacity limits. The team's final solution consisted of an intuitive frontend visualization, intelligent backend optimization logic, and structured data storage, all developed with a flexible architecture. By utilizing Scrum methods through the project, the team was able to iterate development, receive frequent stakeholder feedback, and gradually improve team technical abilities as well as organizational processes. Also by doing tests, reviewing code and making improvements the team improved the quality of the code, made it easier to maintain, increased system's reliability and enhanced teamwork during the development process.

Eventually the team's solution addressed the problem by changing a manual scheduling process into an automated optimization algorithm. The optimizer schedules when to produce heat to meet the heating demand while keeping the operational costs low and maximizing profits in the electricity market. The application lets users easily see detailed optimization data, configure production units to their liking, export data and check detailed optimization data. Also one of the needs for storing CSV results was changed while developing the project. The team found that using JSON for storage was more flexible and easier to handle for what we wanted to achieve. This allows the system to reduce human effort in creating schedules, minimize error, and efficiently allocate resources. Overall, this project proved that an automatic optimization tool can be helpful for planning operations in district heating systems.

Even though main goals have been achieved, there are still some parts of the project that were not completely finished or are still somewhat limited. The current version of the system relies on local JSON and CSV files rather than relational databases or distributed systems. Some technical debt stayed in certain areas of the code because of limitations in the framework and fast development cycles. Using Avalonia and LiveCharts for the frontend brought up some issues with keeping the code easy to manage, making the user interface responsive, and ensuring good performance when displaying graphics. Moreover, testing methods were brought in quite late during the development process, which led to uneven test coverage throughout the system. There were still some problems related to Scrum that had not been fixed. These problems included not judging how long tasks would take accurately and issues with team members not showing up and slowdowns in the work process during several sprints.

Future efforts could aim on improving both technical setup and usability of the app. A future version of the system might move to a more advanced setup that uses relational databases, cloud services and distributed systems. This would help it handle bigger datasets and support many users at the same time. The optimizer can be improved by adding more different parameters to take for optimization, better ways to manage environmental goals like CO2 emissions and the ability to run on multiple cores for enhanced performance. Extra features like user login, authorization, support for multiple languages, real-time heat map display, better reporting tools and improved scenario management could make the system even more useful. Future development should focus more on automated testing, continuous integration and building a frontend architecture that is easy to maintain. This will help make the system more reliable and make sure it can grow over time.

References

- [1] “Discord.” [Online; accessed 28. May 2026]. [Online]. Available: <https://discord.com>
- [2] Atlassian. “Unlock your team’s best work with Jira Software.” [Online; accessed 28. May 2026]. [Online]. Available: <https://jira.atlassian.com>
- [3] Atlassian. “Capture, organize, and tackle your to-dos from anywhere.” [Online; accessed 28. May 2026]. [Online]. Available: <https://trello.com>
- [4] “Excalidraw | Online whiteboard collaboration made easy.” [Online; accessed 28. May 2026]. [Online]. Available: <https://plus.excalidraw.com>
- [5] “Figma: The Collaborative Interface Design Tool.” [Online; accessed 28. May 2026]. [Online]. Available: <https://www.figma.com>
- [6] “Avalonia UI.” [Online; accessed 28. May 2026]. [Online]. Available: <https://avaloniaui.net>
- [7] “Home - LiveCharts2.” [Online; accessed 28. May 2026]. [Online]. Available: <https://livecharts.dev>
- [8] AvaloniaUtils. “DialogHost.Avalonia.” [Online; accessed 1. Jun. 2026]. [Online]. Available: <https://github.com/AvaloniaUtils/DialogHost.Avalonia>
- [9] “NUnit.org.” [Online; accessed 28. May 2026]. [Online]. Available: <https://nunit.org>
- [10] “PostgreSQL.” [Online; accessed 28. May 2026]. [Online]. Available: <https://www.postgresql.org>
- [11] “MySQL.” [Online; accessed 28. May 2026]. [Online]. Available: <https://www.mysql.com>
- [12] “GitHub · Change is constant. GitHub keeps you ahead.” [Online; accessed 28. May 2026]. [Online]. Available: <https://github.com>
- [13] gizul25. “se2 - Pull Requests.” [Online; accessed 30. May 2026]. [Online]. Available: <https://github.com/gizul25/se2/pulls?q=is%3Apr>

-
- [14] gizul25. “se2 - Maintenance rework #25.” [Online; accessed 30. May 2026]. [Online]. Available: <https://github.com/gizul25/se2/pull/25>

AI Usage

During the project AI tools were used to help out. The main AI tool used was ChatGPT to check report text, especially Abstract and Scrum Implementation. Early in the report, these sections used ChatGPT for corrections, but later they were fully rewritten without the use of AI. The AI tools were also used for review and feedback thus making the report easier to read and more consistent, not to create technical information or sway any conclusions.

Appendix A

Team Contributions

List of precise contributions of each team member for both the report and the development part.

David Avramov

Code

Frontend

Revamping colors for the OptimizerViewModel page, adding additional attributes to OverviewViewModel page netcost graph, GUI tests for ViewModels, revamped the Avalonia Headless tests.

Backend

Initial implementation of Scenario 1 Optimizer and RDM with CSV implementation, implementation of the Backend Overview Page, initial implementation of the Maintenance section in the Optimizer, support for the Optimizer for Scenario 2, Unit tests for AM, Optimizer and OverviewViewModel, fixed profit/revenue confusion in Optimizer logic.

Pull Requests

- RDM implementation #6
- Optimizer implementation #12
- Initial implementation of overview page(Incomplete) #19
- Optimizer maintenance mechanisms implementation #22
- Additional implementation in Optimizer #27
- Added AM unit tests #35
- GUI tests - ViewModels #42
- Revamping graph colors #43
- Optimizer profit fix #44

Pull Requests Reviewed

- Demo 05 18 #34
- Fix optimizer #37
- even more detailed optimizer charts #38
- Production units page fix #40(along with Gintaras)
- improve gitignore #39
- better optimizer error handling #45

Report

Wrote Data Management and Sprint 6. Made corrections and fixed errors to my sections.

Report Reviewed

Reviewed Backend and Front Implementation, Overall technical architecture, Sprint 3. Smaller reviews in other sections(not as thorough). Reviewed the Graphics for Sprint 6 in the LaTeX repo PR.

AI

No AI usage on my part with contributions. From what I am aware, AI was only used for corrections in the initial Scrum Implementation, Abstract and parts of the class diagram text. They were re-written in the final version of the report to not include any AI usage.

Gintaras Zulys

Code

Frontend

Main navigation layout implementation, production units and configuration page Figma prototype, frontend optimizer page implementation, backend optimizer page implementation, better GUI design based on feedback, implemented redesigned GUI. Chart-related changes: layered charts in overview page, dynamic colors from AM, added customizing production unit colors for units, added support for time scale, Added more diagrams for optimizer transparency decision-making. Correct handling of multiple scenarios and periods changing for UI.

Backend

SDM implementation & testing, optimizer scenario JSON file loader, added scenario JSON files for all scenarios. Optimizer proper maintenance support, more thorough tests and slight refactoring. Wrote better optimizer error handling (popup, progress bar, catching errors).

RDM rewriting to support charts (Modified many parts, including DM, optimizer, RDM and optimizer page) and exports in JSON. Code refactoring and cleanup, expanded RDM to support storing multiple scenarios and periods.

Pull Requests

- SDM implementation #1

- Sdm testing #3 (changes are made by me but PR opened by Leon with his fixes)
- implement gui navigation layout #4
- add more SDM testing #7
- Frontend optimizer #8
- implement backend optimizer #13
- Rdm charts #15
- scenario JSON, dynamic period update #16
- scenario data AM #20
- scenario 2 data #23
- add layered diagram with dynamic colors from AM #24
- implement improved GUI design #28
- Refactoring & cleanup #30 (closed and merged in another PR with merge conflicts resolved)
- more scenarios #31
- production unit color #32
- Demo 05 18 #34 (cleanups & other PRs with merge conflicts resolved)
- Fix optimizer #37
- even more detailed optimizer charts #38
- improve gitignore #39
- better optimizer error handling #45

Pull Requests Reviewed

- RDM implementation #6
- Data manager #11
- Backend product units page #18
- Maintenance rework #25

- Additional implementation in Optimizer #27
- Added unit images to production units page #29
- maintenance PoC #33
- Added AM unit tests #35
- Side panel improvement #36
- Production units page fix #40
- Diagrams fix 05 25 #41
- GUI tests - ViewModels #42
- Revamping graph colors #43
- Optimizer profit fix #44
- End-to-End #46

Report

Wrote Sprint 1 report, overall technical architecture, frontend implementation, backend implementation, refactoring, code review, sprint implementation (sprint plan), overall group reflection, references.

Many small fixes throughout the whole report based on feedback from Alan and Adam - making text more formal, slight reordering, fixing, grammar. Created custom sprint backlog graphics for all Jira screenshots from sprint planning and reviews, for more readability.

Report Reviewed

Reviewed all changed parts of the report continuously with many changes suggested. It includes Sprint 2, Sprint 5, Sprint 6, package diagram, discussion sections, Abstract, Scrum Implementation, Conclusion, AI usage and many more. Reviewed many LaTeX PRs and have reviewed the entire report multiple times.

AI

The only use of AI disclosed to me was by Nadia, who used corrections for early revisions of Abstract, Scrum Implementation and a part of class diagram text. All relevant sections were fully rewritten manually in the final version without use of AI to my knowledge.

Leon Jürgen Thye

Code

Frontend

Backend Connection of Production Unit Page, Moving of configuration page into Production Unit Page, Production Unit Page, mostly reviewing. Project structure updates, early cleanups.

Backend

Creation of DataManager, Backend connection to Frontend of Production Unit, mostly reviewing. Project structure updates, early cleanups.

Pull Requests

- Data manager #11
- Backend product units page #18
- Maintenance rework #25

Pull Requests Reviewed

- SDM implementation #1
- Sdm testing #3
- implement gui navigation layout #4

- implement Am, populate Asset folder #5
- RDM implementation #6 (together with Gintaras)
- add more SDM testing #7
- Frontend optimizer #8
- Configuration page layout #9
- Optimizer implementation #12
- implementation backend optimizer #13
- wip: unfinished production units frontend #14
- Rdm charts #15
- scenario JSON, dynamic period update #16
- Initial implementation of overview page(Incomplete) #19
- scenario data AM #20
- Optimizer maintenance mechanisms implementation #22
- scenario 2 data #23
- add layered diagram with dynamic colors from AM #24
- add heatgrid map to overview #26

Report

Written Sprint 2 first drafts got helped by Gintaras for finishing it. Written some of the first functional and nonfunctional requirements, moved functional and nonfunctional requirements into LaTeX for the report Appendix, moved Meeting-Logs to into LaTeX for the report Appendix, written my contribution, written Authorship and Reviews for Appendix.

Report Reviewed

Just done one LaTeX based Review

AI

Non AI use of my part in all of my contributions. And I did not know of any AI usage in the project.

Nadia Kristensen

Code

Frontend

Frontend overview page, fixed the diagrams slightly like for adding the legends for each diagram and the axis units, adding a layered diagram for the costs, adding electricity production and consumption graph and few more

Backend

Backend configuration page, End-to-End testing

Pull Requests

- frontend-overview-page (Manual Merge)
- configuration-backend (Manual Merge)
- fixed -diagrams #40
- End-to-End #46
- E2E #47

Report

Wrote Sprint 5, Class Diagram, Sequence Diagram, Package Diagram, Abstract, Scrum Implementation, Conclusion, AI Usage, Testing, Requirements Engineering. Many fixes based on Alan and Adam feedback - fixing the grammar, rewriting some parts, reordering parts.

Report Reviewed

Reviewed most changed parts of the report. It includes Sprint 4, discussion and reflection (Scrum, Refactoring), backend and frontend implementation

AI

I used AI for early revisions for Abstract, Scrum Implementation and a part of class diagram text. Now all sections are fully rewritten without usage of AI.

Oliwia Roksana Moskalik

Code

I designed the Figma prototype for the optimizer page and designed the optimizer algorithm. I implemented the frontend configuration page (no longer in the final product), images for the production units page, and updated visuals for the side navigation panel.

Pull Requests

- frontend-configuration #9
- unit-icons #29
- side-panel-improvements #36

Code Review

I reviewed the implementation of improved GUI design, addition of production unit colours and graph colours improvement.

- Implement improved GUI design #28
- Production unit color #32
- Production units page fix #40

Report

I set up the document for the report, LaTeX repository and later transferred most of the report to LaTeX. I wrote Problem Statement, Sprint 4, Activity Diagram, and Scrum Discussion. I wrote and added Group and Supervisor contracts. I wrote the majority of the functional and non-functional requirements and meeting logs up to Sprint 5, which were both later transferred to the LaTeX version of the report by Leon. Lastly, I corrected and improved grammatically many sections, both based on Alan or Adam's feedback and my own reviews of the whole report.

Report Reviewed

I continuously reviewed all changes to the report, both on Google Docs and GitHub. Reviewed PRs included updated report sections based on Adam and Alan's feedback, Sprint 2, adding references to the report, general review of the whole report and many small fixes of the LaTeX version.

AI

During the course of the project no AI was used on my part, all of the code refactoring was done manually, no additional checks were made on my part, so I am not aware of any external usage of AI. There was a discussion about AI usage for correction for some of the report sections (Abstract, Scrum Implementation and class diagrams), which got rewritten later on to not include AI generated text, but as far as I am aware it was not used for the code itself, or wasn't raised during discussion.

Yaroslav Savenko

Code

Frontend

Mainly production units page implementation including further additional fixes, added visual grid representation to the overview page.

Backend

AM implementation.

Pull Requests

- Add local json to AM #2
- implement AM, populated Asset folder #5
- wip: unfinished production units frontend #14
- add heatgrind map to overview #26
- maintenance PoC #33
- Production units page fix #40

Report

- Two reviews of the LaTeX report (Meeting Logs, Sprint 3 correction)
- Made some reviews in the google docs(Abstract, Technical Aspect)

Report Reviewed

- Problem requirements & problem objectives
- Sprint 3

AI

During the course of the project no AI was used on my part, all of the code refactoring was done manually, no additional checks were made on my part, so I am not aware of any external usage of AI. There was a discussion about AI usage for correction for some of the report sections (Abstract, Scrum Implementation and class diagrams), which got rewritten later on to not include AI generated text, but as far as I am aware it was not used for the code itself, or wasn't raised during discussion.

Appendix B

Meeting Logs

Table B.1: Meeting Logs

Date: 02-03-2026
Duration: 10:00 - 12:30
Place: Løkken
Type: Sprint Planning
Attendance: David, Gintaras, Leon, Nadia, Oliwia
Scrum Master: Gintaras
Agenda: Signing group and supervisor contracts

Date: 06-03-2026
Duration: 10:00 - 12:30
Place: Løkken
Type: Daily Stand-Up
Attendance: David, Gintaras, Leon, Nadia, Oliwia
Scrum Master: Gintaras
Agenda: Filling in midterm feedback.

Date: 09-03-2026

Duration: 10:00 - 12:30

Place: Løkken

Type: Daily Stand-Up

Attendance: David, Gintaras, Oliwia, Yaroslav

Scrum Master: Gintaras

Agenda: Presenting Excalidraw, deciding who goes to course retrospective

Team Member	What I've done	What I'll do	Any Impediments
David	RDM(CSV)	RDM implementation	None
Gintaras	SDM implementation & testing (waiting on Leon to integrate)	Start work on main navigation layout GUI implementation.	Waiting for Leon to integrate testing.
Leon	I wrote 9 Functional requirements and 1 Non-Functional requirement in the docs.	Integrate testing	None
Oliwia	Created functional and non-functional requirements, separate from Leon's so we can review them and merge.	Put names on the requirements, after they are reviewed.	I was not sure how detailed the requirements should be.

Minutes: We created notes for the feedback session. We decided to switch the Scrum Master every Sprint. We decided to implement the Excalidraw board into our workflow.

Date: 11-03-2026

Duration: 8:15 - 10:15

Place: Løkken

Type: Daily Stand-Up

Attendance: David, Gintaras, Leon, Nadia, Oliwia

Scrum Master: Gintaras

Agenda: Backlog expansion, Figma prototype expansion, showcasing Excalidraw to others, story points estimates, review user stories and requirements

Team Member	What I've done	What I'll do	Any Impediments
David	RDM and CSV export	Continue on working on the export of the files until Monday	Waiting for the AM
Gintaras	Finished SDM testing, researched how to approach GUI implementation	Implement main nav GUI until Friday	SDM testing integration was messy but it works I think
Leon	Helped at SDM testing, So Assets loading is working	A new task, I am open for code testing and review	None
Nadia	Didn't have time to do anything	A task	None
Oliwia	Waiting for review since Monday	A new task	No review on Monday

Minutes: We expanded the backlog. We decided on Reviews being another task on Jira.

Date: 13-03-2026**Duration:** 11:30 - 14:00**Place:** Løkken**Type:** Sprint Review/Retrospective**Attendance:** David, Gintaras, Nadia, Oliwia, Yaroslav**Scrum Master:** Gintaras**Agenda:** Prepare slides for Monday

Team Member	What worked well?	What didn't work well?	What and how will it be improved?
David	Thanks to the good flow, organization and consistency of the team we managed to reach most of our goals and tasks.	Not the most pleasant experience with assigning tasks in Jira.	More discussion on what each task does and how we can achieve its completion. A possible way to improve is planning poker.
Gintaras	We eventually did deliver what we've expected, the only task left was the one added during the sprint to fill in if there's more time. Integrating reviews is great for awareness and transparency but it is costly on time and delays.	Jira features like teams are not working nicely, split tasks are odd to use, no ability to assign someone to review the task. Our team story point estimate unit didn't match the reality (about twice as slow than our expected speed). There was some uneven distribution of tasks based on story points but looking at it plainly, it is not that bad.	To help with task progression, have more tasks available to be started (in the backlog). Assigning tasks formed as questions and not statements (already started implementation this week)
Nadia	We did what we predicted.	Assigning tasks in Jira didn't work well.	Assigning in Jira.

Oliwia	We did not overcome; we created a solution for decision-making so everyone is included and the process is more transparent; collective reviews are good for awareness.	No review roles, so unclear how to move the task from review to done; sometimes Jira updates came late; throwing tasks at people without asking first; not much progress in the first week of the sprint, despite the tasks having story points equivalent to 1-4 days; JIRA.	For assigning reviews -> we can use labels to assign the reviewer. Better question for stand-ups so there is higher transparency to catch why people are not progressing as estimated: If not progressing as estimated, what is in the way of the progress? Be specific. What was your time allocation for the task since the last meeting?
Yaroslav	Everyone contributed to the final product increment		Making each task have a description.

Date: 16-03-2026

Duration: 10:00 - 12:00

Place: Løkken

Type: Sprint Planning

Attendance: David, Gintaras, Leon, Nadia, Oliwia, Yaroslav

Agenda: Assigning the scrum master, filling product backlog, presentation fixes, sprint review notes for the supervisor, feedback for Amir

Minutes: We decided to move planning poker to the next meeting due to time constraints. Leon is the new scrum master.

Date: 18-03-2026

Duration: 8:15 - 10:15

Place: Løkken

Type: Daily Stand-Up

Attendance: David, Gintaras, Leon, Nadia, Oliwia, Yaroslav

Scrum Master: Leon

Agenda: Progress, story points for Sprint 1, assignment of tasks, Amir meeting questions

Team Member	What I've done	What I'll do	Any Impediments
David	Started working on the Optimizer	Finish it up, review and test it	None
Gintaras	Finished production units and configuration page Figma prototype	Implement frontend optimizer page	None
Leon	Started working on the Data Manager implementation, fixed some nullable warnings in AM	Done with the Data Manager, review and test the code for the other tasks	None
Nadia	Started implementing the frontend overview page	Finish the frontend overview page	None
Oliwia	Figma prototype for the optimizer page.	Frontend configuration page	None
Yaroslav	started implementing production units frontend in Avalonia	Finish the main display page	Some of the back-end not implemented yet(e.g. editing units)

Minutes: We decided to be more specific with our daily stand-ups.

Date: 23-03-2026

Duration: 10:00 - 10:30

Place: Løkken

Type: Daily Stand-Up

Attendance: David, Gintaras, Leon, Nadia, Yaroslav

Scrum Master: Leon

Agenda: Progression

Minutes: Last tasks assigned for the sprint.

Date: 25-03-2026**Duration:** 8:15 - 10:15**Place:** Løkken**Type:** Daily Stand-Up**Attendance:** David, Gintaras, Leon, Nadia**Scrum Master:** Leon**Agenda:** Progression

Team Member	What I've done	What I'll do	Any Impediments
David	Worked on and implemented the Optimizer into the project	Work on the backend for the overview page	None
Gintaras	Looked into livecharts and data bindings thoroughly	Integrate charts and implement backend for optimizer page	None
Leon	I look into data bindings MVVM pattern and experimenting with the DataManager. Reviewed code merged them, restructured the project	Integrating new Optimizer into DataManager, comparing results. Reviewing other code and merging. Implementing the Product Units page Backend.	Potential waiting is needed because of the frontend Product Units.
Nadia	Started working on implementing backend for configuration page	Finish backend of configuration page	None
Oliwia	Finished the tasks (Jira prototype & frontend implementation of Configuration page)	I'll be not available for a week, so I'll continue from the next sprint.	None

Date: 08-04-2026

Duration: 8:15 - 10:15

Place: Løkken

Type: Sprint Retrospective/Sprint Planning

Attendance: David, Gintaras, Leon, Nadia, Oliwia, Yaroslav

Scrum Master: Yaroslav

Agenda: New scrum master, presentation for friday, sprint retrospective, sprint review

Team Member	What worked well?	What didn't work well?	What and how will it be improved?
Gintaras	There was some progress.	Communication and coordination. Meetings weren't well structured and helpful (improvements weren't integrated). There were dependencies between different tickets and there was no coordination between them. The tickets which were moving slowly weren't addressed early and resolved. Quality control mechanisms are not followed well: writing tests, quality PR review (it gets merged with typos, no tests, debugging prints, etc.). Need for redesigning modules arises during writing of other parts of code.,	When you do something, write about it on Discord after you do it without waiting for a meeting. If you're not feeling comfortable to talk about your issues, talk to the Scrum Master first. Scrum Master's tasks will be defined. Upload code even if not finished, because issues are then easier to solve as others can check it out. If you think your task needs more descriptions, just send it on Discord and others will help.
Leon	The implementation of Frontend was mostly great	Communication and coordination for tasks with dependencies.	

Nadia	Most tasks were done.	Lack of communication	
Oliwia	First week progressed well. The tasks were clear and distributed. - no comments on the second week due to absence.	The goal was a bit too ambitious considering the approaching holidays. Additionally, the lack of Review on Friday hindered the accountability and progress as people didn't have the pressure to finish assigned tasks. The sprint retrospective was also moved. Last sprint's improvements were not fully implemented.	
Yaroslav	Tasks were distributed and there was some progress made on each of them	Mostly communication.	

Date: 13-04-2026**Duration:** 10:00 - 12:30**Place:** Løkken**Type:** Daily Stand-Up**Attendance:** David, Gintaras, Oliwia, Yaroslav**Scrum Master:** Yaroslav**Agenda:** Midterm presentation

Team Member	What I've done	What I'll do	Any Impediments
David	Due to reasons outside of my control I wasn't able to add anything of note since the last meeting.	I'll be working on implementing the back-end overview page	None
Gintaras	RDM rewriting to support charts. Modified many parts, including DM, optimizer, RDM and optimizer page. Now RDM exports data in JSON file, can support storing both summary data such as total cost but also row data like scheduler data which was not possible with CSV before. Swapping between different production units works too.	I'll finish up the RDM rewrite and integrate the optimizer frontend page with RDM data.	Will send a message to Amir and that will take time
Oliwia	Thought/designed how to make optimizer algorithm, checked out more LaTeX	What is there to do? / I can begin adjusting the report template to our project.	None

Yaroslav	Worked on Production Unit Frontend(additional view + styling adjustments)	Hopefully wrap up.	None
----------	---	--------------------	------

Minutes: We started working on the presentation for Monday.

Date: 15-04-2026

Duration: 8:15 - 9:15

Place: Online

Type: Daily Stand-Up

Attendance: David, Gintaras, Leon, Nadia, Oliwia, Yaroslav

Scrum Master: Yaroslav

Agenda: Progression

Team Member	What I've done	What I'll do	Any Impediments
David	Worked with the optimizer.	I'll publish the optimizer branch.	None
Gintaras	RDM export	Working on scenario json file and dynamic period updating	None
Leon		I'll finish the backend for production units today	None
Nadia	Configuration backend page		Configuration backend is not working, but I don't know why.
Oliwia	Adjusted report template for our project.	I'll figure out pdflatex so I can send the template and we can all collaborate on it.	Waiting for tasks and updates, because I don't know what there is to do.
Yaroslav	PR		None

Minutes: Online meeting on voice chat did not fully work, so it was instead held through text messages.

Date: 17-04-2026**Duration:** 11:30 - 14:00**Place:** Løkken**Type:** Retrospective**Attendance:** David, Gintaras, Oliwia, Yaroslav**Scrum Master:** Yaroslav

Team Member	What worked well?	What didn't work well?	What and how will it be improved?
David	More transparency between teammates compared to the last two Sprints.	The distribution of tasks between members wasn't as consistent contrary to the member's abilities, which resulted in slower progress.	Conversations between members outside of Daily Standups will enhance teamwork and improve transparency.
Gintaras	Communication slightly improved. We did make some progress.	Tasks are very slowly progressing, ignored deadlines, a lot of tasks piled up at the end of the sprint, the meetings were unstructured.	More frequently asking others if they need help. Moving the meeting in case of disturbances.
Oliwia	We implemented some actions from the last retrospective and it turned out working well. I think people are a bit more comfortable with speaking up, so there is more psychological safety.	Not a proper start of the sprint, the sprint planning was late and rushed. We also agreed on a deadline on the tasks from the previous sprint, but it was ignored. Personally, I was quite lost due to the unproper planning as I did not have a task from the last one. I also think that sometimes the meetings are not as productive as they should.	If you go into a discord meeting, say "HI" and do not mute your microphone. If someone comes in, say "HI". If not time for a meeting, move the meeting.

Yaroslav	More communication improvements, more elaborate attempts towards working around issues	A lot of tasks accumulated from the previous sprint.	Live Online meetings require more responsiveness.
----------	--	--	---

Date: 20-04-2026

Duration: 10:00 - 12:00

Place: Løkken

Type: Sprint Planning

Attendance: David, Gintaras, Leon, Nadia, Oliwia, Yaroslav

Scrum Master: Oliwia

Agenda: Figuring out what there is to do, populating product backlog

Minutes: We created all the tasks that are left. Everyone estimated how much time (in story point) they have during this sprint so that tasks can be assigned accordingly.

Date: 22-04-2026

Duration: 8:40 - 10:00

Place: Løkken

Type: Daily Stand-up

Attendance: David, Gintaras, Oliwia

Scrum Master: Oliwia

Agenda: Report decisions, scenario support decisions, report tasks

Team Member	What I've done	What I'll do	Any Impediments
David	Nothing, worked on the AOP homework for the last couple of days		None
Gintaras	Scenario 2 data	Add layered chart in overview page	Some architecture re-work will need to be done for application to support multiple scenarios properly
Leon	Because of AOP I am not started yet with the configuration page and optimizer		None
Oliwia	I created tasks based on the new template. I set up LaTeX, and organized space on Google Docs for the report writing.	Write (at least rough) problem statement.	None
Yaroslav	I have already finished adding the heating grid map to the overview page.		The graphs do not seem to align within the scrollview, and instead scale within the window itself

Minutes: We decided to work on Google Docs and then at the end move everything to LaTeX. Every Scrum Master writes about their own Sprint in the report. We re-estimated points for the report tasks.

Date: 24-04-2026**Duration:** 8:40 - 10:00**Place:** Løkken**Type:** Daily Stand-up**Attendance:** David, Gintaras, Nadia, Oliwia, Yaroslav**Scrum Master:** Oliwia**Agenda:** Progression

Team Member	What I've done	What I'll do	Any Impediments
David	Due to AOP and Maths I didn't do much	I'll actually work on stuff I swear - Optimizer Scenario 2	None
Gintaras	Added layered diagram with dynamic colors from AM, fixed legends, added support for time scale.	I'll take rest until Monday and start work on new improved UI design (better visually and functionally)	None
Nadia	I did the sequence and class diagram for report	I will focus on package diagram, i could also take on another quick task	Misunderstood about the tasks in the beginning.
Oliwia	I'm finishing up meeting logs for the report's appendix.	Finalize meeting logs and maybe write rough problem statements.	None
Yaroslav	Finished adding heat grid map to the overview page	Probably start working on doing the problem objectives and requirements, maybe take another task	None

Date: 27-04-2026

Duration: 10:00 - 10:15

Place: Løkken

Type: Daily Stand-up

Attendance: David, Gintaras, Nadia, Oliwia

Scrum Master: Oliwia

Agenda: Progression

Team Member	What I've done	What I'll do	Any Impediments
David	Optimization for the Optimizer	Work more on proper Optimization and start Scenario 2 support for it.	None
Gintaras	Since Friday, regaining energy for the work week.	Create a better design for the GUI	None
Leon	I've implemented the moving of configuration page into product units.	If the moving is out of the review, then I start on redoing the optimizer for better optimizing.	Some hurdles about, if there should be just one setting for min maintained hour and max maintained hour or per unit.
Nadia	I have not done anything, didn't have time.	Do the package diagram	None
Oliwia	I finished writing meeting logs	Problem statement	None
Yaroslav		I'll finish the draft for problem objectives and requirements	None

Date: 27-04-2026**Duration:** 08:25 - 08:40**Place:** Løkken**Type:** -**Attendance:** Gintaras, Oliwia, Yaroslav**Scrum Master:** Oliwia**Agenda:** Progression

Team Member	What I've done	What I'll do	Any Impediments
Gintaras	Worked on better design for the GUI	Finish work on better design for the GUI	None
Oliwia	Started on the problem statement, I haven't finished it because of math	Finish the problem statement so it can be reviewed	None
Yaroslav	Working on problem objectives and requirements, probably wrapping up	Start working on sprint 3 report	None

Date: 01-05-2026**Duration:** 12:15 - 13:30**Place:** Løkken**Type:** Retrospective**Attendance:** David, Gintaras, Nadia, Oliwia, Yaroslav**Scrum Master:** Oliwia

Team Member	What worked well?	What didn't work well?	What and how will it be improved?
David	A lot of progress on the tasks	The meeting for the most part were only half the amount of people	Everyone actually going to the daily standups so we have more knowledge and transparency between each other. Also for Wednesday instead of starting it at 8:15 we could do it at 9:00
Gintaras	Things seem to be progressing	Agreed with Oliwia	Add "fin" to our meetings (clear ending), Wednesday meetings to start at 9:00
Nadia	Lots of tasks were done or in review	Meetings haven't been good in progress. Lack of communication.	Maybe set a kind of a template for a meeting how it should go and set meetings on Wednesday for 9:00
Oliwia	Things seem to be progressing. It was useful that we did point estimations for everyone, so people were more inclined to follow the workload they predicted themselves.	Meetings seemed optional, and time for the meeting was treated more like a suggestion.	Wednesday's meeting moved to a later hour.
Yaroslav	Everyone finished at least one task.	Not many people were present during daily standups	Maybe cutting the time after the daily standup has concluded, but if anyone has questions or feel like working on their stuff they can sta

Date: 05-05-2026**Duration:** 15:00 - 16:30**Place:** Løkken**Type:** Sprint Planning**Attendance:** David, Gintaras, Nadia, Yaroslav**Scrum Master:** Nadia**Agenda:** New scrum master, sprint planning, sprint goals**Date:** 11-05-2026**Duration:** 12:30 - 13:30**Place:** Løkken**Type:** Daily Stand-up**Attendance:** David, Gintaras, Nadia, Oliwi**Scrum Master:** Nadia**Agenda:** Prepare for demo presentation, daily standup

Team Member	What I've done	What I'll do	Any Impediments
David	Love me some maths	Report and Optimizer fixing	None
Gintaras	Math and a bit of implementing UI work	implementing improved UI	None
Nadia	I wrote the abstract part for the report.	I will finish the other tasks: sequence diagram and write scrum implementation for report.	None
Oliwia	Math.	Sprint 4 and pics for components	None

Date: 13-05-2026

Duration: 9:00 - 10:00

Place: Løkken/Online

Type: Daily Stand-up

Attendance: David, Gintaras, Nadia, Oliwia

Scrum Master: Nadia

Agenda: Daily standup

Team Member	What I've done	What I'll do	Any Impediments
David	Polishing the Optimizer	Finishing up the Optimizer and doing my part on the Report	None
Gintaras	implemented re-designed GUI, wrote sprint 1 report	write technical documentation for the report	None
Nadia	Updated the sequence diagram	scrum implementation	None
Oliwia	Review, worked on the images but run into issues and started the Sprint 4 for report	images and finish the Sprint 4	With image implementation

Date: 15-05-2026

Duration: 12:30 - 14:45

Place: Software Hub(Alsion)

Type: Retrospective

Attendance: David, Gintaras, Oliwia, Yaroslav

Scrum Master: Nadia

Team Member	What worked well?	What didn't work well?	What and how will it be improved?
David	Things are coming together nicely(tasks, communication and transparency).	Slower progress and less attendance due to external factors(specifically the Math 2 Exam and to a lesser degree the 4th AOP homework), reviewing issues, most if not all meetings only had a fraction of the team present both physically and online(most likely for the same reasons as the former).	While the communication has certainly improved it still needs more time to improve(some members are still shy to talk about their progress and issues, this could be improved by focusing on a more ethical discussion between members to give more of a sense of physical safety). Attendance needs a serious discussion between the team to find a solution on low attendance(If 2 or more people aren't able to come we can change the time of the meetings to be in a better time frame - both online and physically).

Gintaras	Some things got done.	Slower progress than expected. Tons of reviews which get pushed back to in progress and some tickets are from last sprint which haven't been finished this sprint. Avg attendance only 3-4/6 people in a meeting. A bit of an issue that contract to attend and notify a day before if can't attend is not followed or able to enforce properly. Scrum Master absent at times. Half of team members didn't meet their estimated work time by quite a lot.	Long reviews by decreasing review latency. Reminder such as "Review is required for everyone and report will reflect review efforts" could work. For attendance, sending confirmatory message a day prior if they will attend meeting at X time in Y place. Expect that some members might be online regardless and need to prepare for hybrid meetings.
Oliwia	Communication about done tasks on discord is working well, progress on tasks is alright. Work load was adapted to the math exam week, so it was not overestimated. More elaborate answers for questions.	Scrum master absent at times, many things in review (people not reviewing, slow follow up) even from previous sprint. Meetings are not moved when people cannot attend because the information about absence is at night/morning/few min before the meeting when there is no time to reschedule it properly. Contracts are not followed.	Rescheduling meetings if people are not attending, so everyone agrees on the time; Some kind of deadlines/tokens for the review of the report.

Yaroslav	The report part is going smoothly.	Lot of tasks are accumulating in review, not many people being present due to external factors(math exam, aoop, communal maintenance).	Improving attendance, agreeing on time before meeting way before in advance, doing a poll on who's going to attend.
----------	------------------------------------	--	---

Date: 18-05-2026

Duration: 9:30 - 11:50

Place: Løkken

Type: Sprint Planning

Attendance: David, Gintaras, Nadia, Oliwia, Yaroslav

Scrum Master: David

Agenda: New scrum master, sprint planning, sprint goals, discussing finishing the project for this Sprint

Date: 19-05-2026

Duration: 12:00 - 15:00

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Oliwia, Yaroslav

Scrum Master: David

Agenda: Progress, working, polishing the code and adding the rest of the unit tests

Team Member	What I've done	What I'll do	Any Impediments
David	Made unit tests for the AM	Writing the backend section of Implementation in the Report	None
Gintaras	Code refactoring and cleanup, expanded RDM to support storing multiple scenarios and periods, correct refreshing behavior for UI on scenario and period change, added 2 more scenarios, added customizing production unit colors for units. Wrote technical architecture report part. Reviewed all changed parts of the report, production units image PR and scenarios panel update PR. Added more diagrams for optimizer transparency decision-making.	Finish up work on fixing the optimizer.	None

Oliwia	Added production unit images, wrote the Sprint 4 section for the report, updated visuals for the side panel, reviewed PR and changes in report, and created the action diagram for the report.	Write the text for the action diagram for the report.	None
Yaroslav	Refining production units page, mostly visual improvements	Fixing the crash after adding or removing production unit, implementing check for production unit consumption resource	None

Date: 20-05-2026

Duration: 12:00 - 15:45

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Oliwia, Nadia

Scrum Master: David

Agenda: Progress, working

Team Member	What I've done	What I'll do	Any Impediments
David	Swapped my task(the backend implementation) with the data management one and started working on it	Fully finishing the data management section of the report and polishing it and start Sprint 6 when we are almost done with the Sprint	None
Gintaras	More detailed charts for optimizer, proper maintenance support and slight refactoring. Added more thorough tests for optimizer. Reviewed sprint 5 and package diagram.	Write backend implementation part for the report	None
Oliwia	Transferred and formatted the parts of the report that we have into LaTeX, including images.	Finish activity diagram text for report.	None
Nadia	I have modified the Sprint 5 and finished working on Package Diagram.	Fix the diagrams	None

Date: 21-05-2026

Duration: 12:00 - 15:00

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras

Scrum Master: David

Agenda: Progress, working

Team Member	What I've done	What I'll do	Any Impediments
David	Writing the Data Management in the Implementation part of the Report and reviewing the rest of the report	Elaborate on the Data Management part and continue reviewing, possible implementation of GUI testing - ViewModel(depends on the person who does it at the end)	None
Gintaras	Wrote frontend and backend implementation parts for the report. Added more detailed charts for the optimizer. Reviewed package diagram again.	Unknown as I don't have any tasks assigned anymore.	Lack of tasks in sprint and product backlog

Date: 22-05-2026

Duration: 9:15 - 13:25

Place: Løkken

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Yaroslav, Nadia, Oliwia

Scrum Master: David

Agenda: Progress, working

Team Member	What I've done	What I'll do	Any Impediments
David	Glancing over the report	Do the GUI Test: ViewModels, put more stuff in Data Management in the Report	None
Gintaras	Thoroughly reviewing LaTeX entire report, improved frontend implementation part, discussing a part for the report	Fix up LaTeX version once it has its own repo, update UML, write reflection section of the discussion section.	Just waiting for LaTeX report repo to be uploaded.
Nadia	Started fixing the diagrams	Finish fixing the diagrams	None
Yaroslav	Trying to wrap up production units page overhaul.	Start working on End-2-End tests	None
Oliwia	Reviewed the report, activity diagram report section, discussion for the report	Create a repository for the LaTeX version of the report, write the SCRUM part of the discussion for the report.	None

Date: 25-05-2026

Duration: 12:10 - 15:00

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Yaroslav, Nadia, Oliwia

Scrum Master: David

Agenda: Progress, discussion for the report

Team Member	What I've done	What I'll do	Any Impediments
David	Report discussion, started working on the Sprint 6 part of the report	Continue working the Sprint 6, fix and add GUI tests	None
Gintaras	Report discussion, wrote refactoring and code review report parts	Review 2 PRs	None
Nadia	Report discussion, fixed package diagram, worked on requirements engineering and updated scrum implementation	Finish requirements engineering and update fixed diagrams text for report	None
Yaroslav	Report discussion, finished work on fixing the production unit page	Make additional changes to production units page based on PR feedback, start working on End-to-End testing	None
Oliwia	Report discussion, organized notes for Scrum section of report discussion	Finish the Scrum discussion section for the report	None

Date: 26-05-2026

Duration: 12:00 - 15:15

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Nadia, Oliwia

Scrum Master: David

Agenda: Progress, discussion for the report

Team Member	What I've done	What I'll do	Any Impediments
David	Fixed the GUI testing errors(refactored it entirely) and implemented GUI View-Model tests(pain)	Continue working on the Sprint 6 in the Report	None
Gintaras	Reviewed diagrams, production units, and slight LaTeX changes PRs. I have added support for KPIs to change based on selected unit or all units selected in the optimizer page and a few additional diagram tweaks. Implemented a lot of feedback from Alan and Adam - impersonal in non-sprint texts, frontend diagram, slight re-ordering and fixes. Created graphics for more compact and readable sprint planning and review figures in the report.	Write overall reflection for the report.	None

Nadia	Fixed scrum implementation, diagrams parts. I have written Testing in discussions and reflections, AI usage, finished requirements engineering. I have also reviewed backend and frontend implementation and Refactoring for the report.	I will work on another task like writing the Conclusion part for a report.	None
Oliwia	I have implemented some of the feedback from Adam, I reviewed a few report PRs, and I have finished the Scrum discussion section.	I will fix the activity diagram.	None

Date: 27-05-2026

Duration: 12:00 - 15:30

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Nadia, Oliwia, Leon, Yaroslav

Scrum Master: David

Agenda: Progress, discussion for the report

Team Member	What I've done	What I'll do	Any Impediments
David	Reviewed Discussion and Reflections in the Report, revamped the colors in the graphs for the Overview and Optimizer pages in the project	Continue reviewing and working on Sprint 6 part for the Report	None
Gintaras	Wrote overall group reflection, reviewed GUI testing and graph colours PRs. Reviewed new discussion sections, Abstract, Scrum Implementation, Conclusion parts of the report. Tweaked new Scrum implementation as part of the report review.	Polish up what's remaining, review anything new what pops up, add graphics for tasks for new report sections.	None
Nadia	I have changed both Scrum Implementation and Abstract, i have worked on Conclusion part, i have reviewed Discussion and Reflections "Scrum"	Finish Conclusion part	None

Oliwia	Reviewed updated sections of the report [Abstract, Scrum Implementation, Discussion, AI Usage] both on docs and as PRs, restructured Solution chapter in LaTeX.	Fix activity diagram, continue reviewing report PRs, and any tasks left for compilation of the project.	None
Leon	None Because of Sickness	Finish Sprint 2 for the Repor	None
Yaroslav	Started implementing suggested changes to production units page overhaul	Start working on E2E tests	None

Date: 28-05-2026

Duration: 12:00 - 15:00

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Nadia, Oliwia, Leon, Yaroslav

Scrum Master: David

Agenda: Progress, report(docx and LaTeX)

Team Member	What I've done	What I'll do	Any Impediments
David	Started on fixing profit naming for the Optimizer, Writing the Sprint 6 section(Everything that is possible to write before 29.05)	Do the Sprint Retrospective for Sprint 6 and finish the profit naming	None
Gintaras	Reviewed graph colors PR, done references page for the report, updated my sections based on Adam feedback. Reviewed changed conclusion and AI usage parts. Reviewed and heavily tweaked sprint 2 report. Added new graphical figures for sprint 2 and sprint 6	Write sprint introduction.	Production units page seems to be stuck, is in progress for 1.5 weeks now.
Nadia	Updated Testing, Conclusion, AI Usage, Scrum, changed small correction from Adam's feedback - Scrum Implementation and Abstract		None

Oliwia	I checked the consistency between docs and LateX, and moved missing parts. I fixed small corrections from Adam's feedback in the report sections. I reviewed report PRs [graph colors, updated report sections based on the feedback], Sprint 2 and Sprint 6 sections, conclusion and AI usage.	Update activity diagram, and continue to review the repor	None
Leon	Finished first draft and second draft of my sprint 2, thanks to Oliwia and Gintaras for Correcting for Merge.	Started working on moving functional requirements and non functional requirements tables to LaTeX format from docs.	None
Yaroslav	Still working on production units page fix, started working on E2E tests	Wrap up production units page fix, focus on E2E tests	None

Date: 29-05-2026

Duration: 12:00 - 13:49

Place: Løkken/Online(Hybrid meeting)

Type: Daily Stand-Up & Working Session

Attendance: David, Gintaras, Nadia, Oliwia, Leon, Yaroslav

Scrum Master: David

Agenda: Progress, report(docx and LaTeX)

Team Member	What I've done	What I'll do	Any Impediments
Nadia	Reviewed changed parts of report, working on End-to-End	Finish End-to-End, review report so all is good	None
Oliwia	Fixed activity diagram, went over the report to check consistency, fixed Scrum challenges based on the feedback, reviewed all changed parts of the report,	Do a final review of the report and optionally fix margin issues.	None
Leon	Finished Moving functional requirements and non functional requirements tables to LaTeX. Started with moving the meeting logs to LaTeX, finished with moving until the meeting from 28.5.26.	Adding the Remaining Meetings until June 1.	None
David	Reviewed Unit production PR, started working of fixing optimizer PR	Fix Optimizer revenue and expenses and Finish Sprint 6	None

Gintaras	Wrote sprint implementation (sprint plan). Reviewed small and suggested small changes for testing and AI usage. Reviewed production units PR, profits/expenses PR, lots of small review LaTeX PRs. Fixed references and slight formatting.	Write better optimizer error handling, make graphics for sprint 6.	None
Yaroslav	Tried to wrap up the production units page fixes, worked on E2E testing, fixed report parts for Sprint 3 and appendix reference	Make final adjustments to production units page, finish E2E tests	None

Date: 29-05-2026

Duration: 13:49 - 16:10

Place: Løkken/Online(Hybrid meeting)

Type: Retrospective

Attendance: David, Gintaras, Nadia, Oliwia, Leon, Yaroslav

Scrum Master: David

Team Member	What I've done	What I'll do	Any Impediments
David	The workflow was decent and most of the tasks were done on time or at least in the time frame of the sprint. Collaboration between members was also really strong this Sprint around.	One issue was that since this was the last Sprint task just kept piling one over another adding pressure to the team and while the hybrid meeting were good for the most part an issue persists where communication wasn't as good compared to physical meetings and led to "ghost calling" situations. Also some tasks took way longer than the assigned time which caused the progress to stall.	We should definitely discuss certain parts to a greater extent(Most notably AI), hybrid meetings while they worked we should stick more to physical ones or at least for the majority to be physically present. There should be a warning when someone late rather than leaving the team in the dark. Everyone should participate in reviewing both for the Pull Requests in GitHub and the Report(Both docx and LaTeX).
Nadia	The work on tasks and collaboration have worked well. Daily meetings helped to finish the tasks faster and keep the flow smooth.	Some tasks took too long regarding the estimated time and this sprint was harder than previous ones.	Discuss everything in detail before so there will be no misunderstandings.

Leon	The daily meetings and the resulting communication are helpful to get the project finished.	Sprints tasks was harder then expected.	Discuss certain topics, Reviews handling, AI. Idea for Reviews, everyone see if there is any reviews open at the start of working session and at the end of it.
Gintaras	The daily work meetings helped with reviewing quickly and pushing the project towards the finish really well. Hybrid format helped with attendance. There was way more communication (during meeting and after) than before due to daily work meetings, countless reviews and feedback. Many tasks were done, and our set deadlines for sending feedback closely matched up. Time availability - no lectures, no AOP homework helped with progress too. Daily standups daily was really good.	Some tasks were stuck, causing slow-down. A team member not disclosing use of AI and only finding out after Adam feedback (though fixed later). Discussing the AI issue of one team member, was pushed as suggestion as 1-1 discussion for more safety but as far as I'm aware it did not happen (most likely because Nadia's texts were changing to be more hand-written). With many small tasks and reviews moving quickly, it was difficult to keep track what is reviewed, what reviews were addressed and what is fully finished.	AI discuss clearly where are the lines for AI for the enxt. and gray areas. Different channels on Discord. Using Excalidraw more for updates, clearer decisions. Many tools, no single source of truth for information, should standardize on Excalidraw.

Gintaras		Some tasks lacked better explanation or examples to reduce amount of reviews required to merge changes. Some meetings were very quiet. Announcing if team members will be on-line, or in-person, or late for the meeting was better but somewhat problematic at times still. Some team members not notifying about their state of progress as well as desired (daily standups was the best source to find out about it and see what to review).	
Yaroslav	The daily standups were helpful, making tasks progress more easily	The most overloaded sprint, some tasks were stuck in the end still, which caused a general slowdown, also late updates to google docs version of report raised more confusion.	Discuss every concern in detail as soon as it arises, make an obligatory participation in review process for everyone, check if there's anything to review at the beginning at ending of the meeting.

Oliwia	The daily meetings helped a lot with progression and better collaboration among the members. Reviews were quick to get resolved. The Sprint Planning went quite well, the remaining tasks were all identified and described well. The progress was consistent through the sprint.	The sprint was quite packed, as a result of poorer progress in the other Sprints. Some members were not so active on the meetings. Docs/LaTeX confussions added more work, as we changed the decision to work on one and move later. Comments with no purpose/no intention to improve towards a member who perhaps was not fully aware of the rules for AI. That AI situation also felt like we were just avoiding the problem, insted of just asking and talking about it as no one followed up on the proposition of discussing it during the meeting / one-on-one. People getting late with no notice / late notice. It felt like the ceremonies were not carried out that well (daily stand-ups were not done together, we just wrote when we remembered and there was no time for saying it in conversation).	Discuss important topics (rules, guidelines) prior, or when they come up; Everyone reviews report PRs and they are not resolved until everyone is done, and have better deadlines for them. Better organization of the Excalidraw board. Have an overview page with FAQ and decisions made.
--------	--	---	---

Oliwia		Reviews on docs were sometimes very late, so someone could not just finish a section, they had to continuously update it (reviews had no deadlines).	
--------	--	--	--

Date: 31-05-2026

Duration: 12:00 - 12:15

Place: Online

Type: Daily Stand-up & Working Session

Attendance: David, Gintaras, Nadia, Leon, Yaroslav

Scrum Master: David

S Agenda: Progress

Team Member	What I've done	What I'll do	Any Impediments
David	Fixed Optimizer profit/revenue issue, finished Sprint 6 section in the Report	Polishing up OptimizerView-Model.axaml(Profits) and polishing the revenue graph in OverviewViewModel, wait for Sprint 6 reviews, write my Contribution to the project	None
Gintaras	Wrote better optimizer error handling, made graphics for sprint 6, reviewed and suggested changes for sprint 6 report. Updated code review section based on feedback. Reviewed E2E testing PR, profits PR and production units PR.	Write my part for contributions and code contributions. Review anything necessary. Read the entire report for mistakes.	Few tasks which I've reviewed are not progressing well.
Leon	Finish moving Meeting-logs. until 29.5. Reviewed code-review-links.	Move this Meeting-Log to LaTeX, Move Contributions based on report structure to LaTeX at midnight, Write My Contributions Part, Write my Ownership Part	None

Nadia	Worked on End-to-End testing	Finish End-to-End	None
Yaroslav	Tried to implement maintenance display for production units page	Finish production units page	Currently the maintained unit is defined during the optimization itself, making it necessary to run the optimizer more than once

Appendix C

Contracts

Supervisor Contract

SDU 2nd Semester Project – Supervisor-Student Collaboration Contract

Department/Programme:	Software Engineering
Semester/Year:	Spring 2026
Project Title:	Heat Production Optimization
Supervisor:	Alan Sieradzki
Student(s):	Oliwia Roksana Moskalik Nadia Kristensen Leon Jürgen Thye Gintaras Zulus David Avramov Yaroslav Savenko
Project Period:	January – May 2026

ECTS: ☐ 10 ECTS

1. Purpose

This contract specifies the collaboration between students and their corresponding supervisor. The agreement sets the rules and expectations from both parties. Following this contract ensures good communication and collaboration throughout the semester project development

2. Expected Student Workload

Each student, to achieve the goal of 10 ECTS points, is expected to commit:

1 ECTS = 27,5 h;

10 ECTS = 10 * 27,5h = 275h

275h / 12 weeks ≈ 23h/week

23 h/week – 4 h/week = 19 h/week

In total 19 h/week for the development of the semester project.

3. Roles and Responsibilities

3.1. Students responsibilities:

- Communicating about any unresolved issues;
- Preparing materials specified for the given deadline;
- Meeting deadlines given for any submission task;
- Working together in a collaborative way with other students in the group;

- Ensuring that everyone is aware of all the modifications, and that the project stays at the similar level of expertise for each student;
- Helping each other if anyone is falling behind;
- Documenting their progress in an Agile environment;
- Writing the report, combining all of the progress and including the given template;
- Maintaining academic honesty and correct referencing;

3.2. Supervisor responsibilities:

- Ensuring good progress of the group;
- Giving a satisfactory answer to any questions students may have in regards to the project;
- Notifying the group about any upcoming deadlines and any given tasks;
- Giving feedback about the progress and the development process, so that the students meet the initial requirements;
- Clearly stating objectives and requirements of the course;
- Supporting the group in case of any issues that might arise and clarifying them the coordinator of the course;

Note: The supervisor is not responsible for any technical related work, writing the report or completing tasks on behalf of the students.

4. Communication

Communication between the parties is expected to be conducted in a respectful manner, ensuring the well-being of all participants.

In the event that any questions arise, students are welcome to contact the supervisor. The supervisor is required to respond within at least two working days after the question has been submitted. For example, if a question is submitted on Friday, the supervisor's response is expected by Tuesday.

The form of communication between the parties must be agreed upon in advance, whether via email or another communication channel.

The same applies to the meeting format: the student and the supervisor must agree on one of the following methods: an online meeting conducted via an agreed-upon communication platform or an in-person meeting at a mutually agreed location.

5. Meetings and Attendance

Both parties are expected to attend student-supervisor meetings at the agreed-upon time. The date and time of each meeting shall be determined mutually by the student group and the supervisor. Meetings should be held on average at least once per week for at most one hour. The format of the meetings shall also be agreed upon by both parties.

Students are expected to attend progress meetings every other week to present updates on their development work. These meetings are scheduled on Fridays from 10:15 to 12:00.

In addition, students are required to attend all mandatory meetings according to the schedule provided on itslearning.

In the event that a party is unable to attend any of the meetings specified above, the other party must be notified in advance, at least one day before.

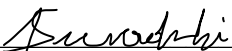
6. Report Writing

The students are responsible for conducting the documentation of their project and writing a report summarizing their development process. The template for the report is given on itslearning platform, where there is a provided description for each mandatory chapter. The template serves as a reference and does not restrict students from adding additional chapters, provided that all specified requirements are met. The encouraged format used for writing the report is LaTeX, which can be created in a shared Overleaf project. However, other formats are acceptable, as long as it fulfills SDU restrictions about an academic report.

The writing phase for the report should start no later than the 20th of April.

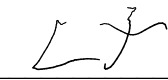
7. Agreement

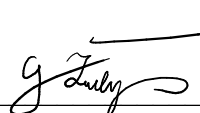
By signing the agreement both parties agree to the given terms, and agree on the following agreed upon rules.

Supervisor name Alan Sieradzki Signature:  Date: 23/02/2026

Student name Oliwia Roksana Moskalik Signature:  Date: 23/02/2026

Student name Nadia Kristensen Signature:  Date: 23/02/2026

Student name Leon Jürgen Thye Signature:  Date: 23/02/2026

Student name Gintaras Zubys Signature:  Date: 23/02/2026

Student name David Avramov Signature:  Date: 23.02.2026

Student name Yaroslav Savenko Signature:  Date: 23.02.2026

Team Contract

SDU 2nd Semester Project

Group Contract

Department/Programme:	Software Engineering
Semester/Year:	Spring 2026
Project Title:	Heat Production Optimization
Student(s):	David Avramov Gintaras Zulys Leon Jürgen Thye Nadia Kristensen Oliwia Roksana Moskalik Yaroslav Savenko
Project Period:	February – May 2026

1. Purpose

This contract specifies the collaboration between team members. The agreement sets the rules and expectations of each member. Following this contract ensures good communication and collaboration throughout the semester project development.

3. Responsibilities

Each member of the group is responsible for:

- Communicating about any issues;
- Meeting deadlines given for any task;
- Working together in a collaborative way with others in the group;
- Replying to messages in 2 working days;
- Maintaining academic honesty and correct referencing;
- Informing the rest of the group if they cannot attend a meeting;
- Informing the rest of the group if they will be late for a meeting;

If responsibilities are not met and no improvement will be made after discussion with the group, the problem will be raised to the supervisor.

4. Communication

Communication between the group members is expected to be conducted in a respectful manner, ensuring the well-being of all participants. Bringing up any difficulties and problems within the project and the group should be encouraged and met with friendliness.

5. Meetings and Attendance

All members of the group are expected to attend all meetings. Two meetings are scheduled every week, on Monday at 10:00 and on Wednesday at 08:15. The duration of the meetings should not exceed two hours. The meetings are primarily in-person, however it is possible to attend the meetings online if necessary.

The meetings are scheduled around SCRUM ceremonies. First Monday of a sprint is the *sprint planning*, and last Friday of a sprint during Software Engineering lecture is the *sprint review* and during supervisor meeting is the *sprint retrospective*.

In the event that a member is unable to attend any of the meetings specified above, the other members must be notified in advance, at least one day before. In case of arriving late to a meeting, the rest of the group should be notified before the start of the meeting. If the group is not notified, the absent or late person is responsible for bringing snacks to the next meeting.

7. Agreement

By signing the agreement each member agrees to the given terms and on the following agreed upon rules.

Student name Olivia Roksana Moskalik Signature:  Date: 02/03/2026

Student name Gintaras Dulys Signature:  Date: 02/03/2026

Student name Nadia Kristensen Signature:  Date: 02/03/2026

Student name Leon Jürgen Thye Signature:  Date: 02/03/2026

Student name David Avramov Signature:  Date: 02/03/2026

Student name Yaroslav Savenko Signature:  Date: 02/03/26

Appendix D

Code Authorship and Review

Authorship and Reviewes per modules:

AM

Author: Gintaras, Yaroslav, Leon, David **Reviewer:** Leon, Gintaras, Oliwia, Yaroslav, David

SDM

Author: Gintaras, Leon, Yaroslav **Reviewer:** Leon, Gintaras

RDM

Author: Gintaras, David, Leon **Reviewer:** Leon, Gintaras

Optimizer

Author: David, Gintaras, Leon, Nadia, Yaroslav **Reviewer:** Gintaras, Leon, David

Data Visualization

Author: Gintaras, Nadia, Leon, Yaroslav, David, Oliwia **Reviewer:** Gintaras, Leon

Optimizer page

Author: Gintaras, Nadia, Leon **Reviewer:** Leon, Gintaras, Oliwia, David

Overview page

Author: Nadia, Gintaras, David, Leon, Yaroslav **Reviewer:** Leon, Gintaras, Oliwia

Production units page

Author: Leon, Yaroslav, Gintaras, Oliwia, Nadia **Reviewer:** Leon, Gintaras, Oliwia

Appendix E

Table of Requirements

Functional Requirements

Table E.1: Functional Requirements

ID	Name	Description
FRH01	Gather Data	The user should be able to gather data from files or databases. Data are hourly informations of heat demand and electricity prices.
FRH02	Gather Assets	The user should be able to gather assets with their specs from files or databases.
FRH03	Displaying Data	The program should be able to display the data to the user, with informative diagrams.
FRH04	Displaying Assets	The program should be to display the assets to the user.
FRH05	Optimizer	The user should be able to configure the optimizer.
FRH06	Display Results	The results from the Optimizer should be displayed, with informative diagrams.
FRH07	Export Results	he user should be able to export the results as like a csv file.
FRH08	Optimizer Gathering	The optimizer needs to get the right data and assets based on the configuration of the user.
FRH09	Optimizer Results	The output of the optimizer is calculated by the assets and data. Maintenance periods for individual assets should be calculated as well.
FR01	Storing Heating-Grid Data (AM)	The system shall store heating-grid configuration data in local files.
FR02	Providing Heating-Grid Data (AM)	The system shall provide heating-grid information (name and graphical representation) to other modules.

FR03	Storing Production Units Data (AM)	The system shall store configuration data for all production units.
FR04	Providing Production Units Data (AM)	The system shall provide name, graphical representation, and operation-point parameters for each unit.
FR05	Storing System Information (SDM)	The system shall store system information (heat demand and electricity prices) in local CSV files.
FR06	Providing Heat-Demand (SDM)	The system shall provide hourly heat-demand time series.
FR07	Providing Electricity-Price (SDM)	The system shall provide hourly electricity-price time series.
FR08	Storing Optimization Results (RDM)	The system shall store optimization results in local CSV files.
FR09	Creating Separate Results (RDM)	The system shall create separate result data for each production unit, which includes produced amount of heat, produced/consumed electricity, production costs, consumption of primary energy and produced amount of CO ₂ .
FR10	Calculating Optimized Schedule (OPT)	The system shall calculate an optimized heat-production schedule for a given period.
FR11	Ensuring Heat Availability (OPT)	The system shall ensure that heat demand is met at any time.
FR12	Compute Net-Production (OPT)	The system shall compute net production costs for each unit for each hour.
FR13	Prioritizing Units (OPT)	The system shall prioritize units based on their net production costs
FR14	Allowing Operation Points (OPT)	The system shall allow units to operate anywhere between 0 and their maximum operation point.
FR15	Supporting Maintenance (OPT)	The system shall support planned maintenance periods (30-60 hours), during which units are excluded from optimization.
FR16	Vizualizing Configurations (DV)	The system shall graphically display the configuration of the heating grid and production units.
FR17	Vizualizing Time Series (DV)	The system shall visualize time series for heat demand, heat production, electricity production/consumption, electricity prices, expenses and profit, primary energy consumption, and CO ₂ emissions.

FR18	Vizualizing Net Production Cost (DV)	The system shall visualize net production costs for all units.
FR19	Including Scenarios	The system shall include Scenario 1 and Scenario 2.

Non-Functional Requirements

Table E.2: Non-Functional Requirements

ID	Name	Description
NFR01	Required Modules	The system shall have following modules: AM, SDM, RDM, OPT, and DV.
NFR02	Static Data Storage	Static configuration data for heating grid and production units shall be stored in local files.
NFR03	Dynamic Data Storage	Dynamic heat demand and electricity price data as well as the optimization results shall be stored in local CSV files.
NFR04	Time Series Resolution	Input time-series resolution shall be 1 hour.
NFR05	Datasets	The system shall support a two week dataset for winter and summer periods.